

Monte Carlo Tree Search

Readings: R&N Ch 5.4 (version 4 only)

Lecture 8: September 28, 2026

Jackie CK Cheung (jackie.cheung@mcgill.ca)

Su Lin Blodgett (sulin.blodgett@mcgill.ca)



Source: Wikimedia
Commons

- Casino of Monte Carlo – what does this have to do with designing game-play bots?

Issues with Minimax and Alpha-Beta

- It's computationally impossible to search the full game, even if we do alpha-beta pruning!
- Therefore, all solutions to chess-sized games rely on having a reasonable evaluation function in practical cases
- It assumes both you and your opponent are playing optimally **with respect to the same evaluation function**

Questions About Minimax

- What if there's some nondeterminism in the game environment?
- What if we don't know the game well enough to **design** a good evaluation function?

Random Simulations

Basic idea:

- The information from a full play-through can be helpful (if imprecise) feedback about the value of states.
- A single run wouldn't be enough, but computers are good at repeating cheap operations many times...

Solution: Random simulation

- Play the game from a state, many times, where the move selection is at least partly based on drawing random numbers.
- Combine the value at terminal outcomes to compute the starting state's value.

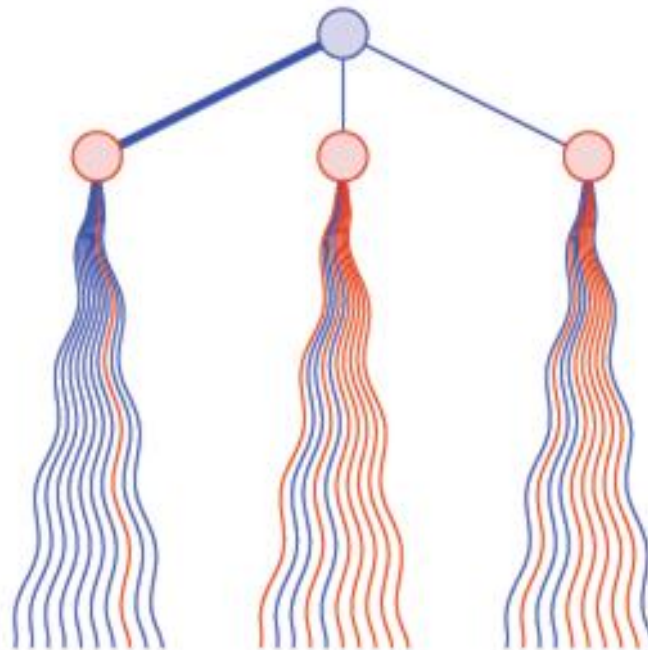
Example

- 3 possible next moves



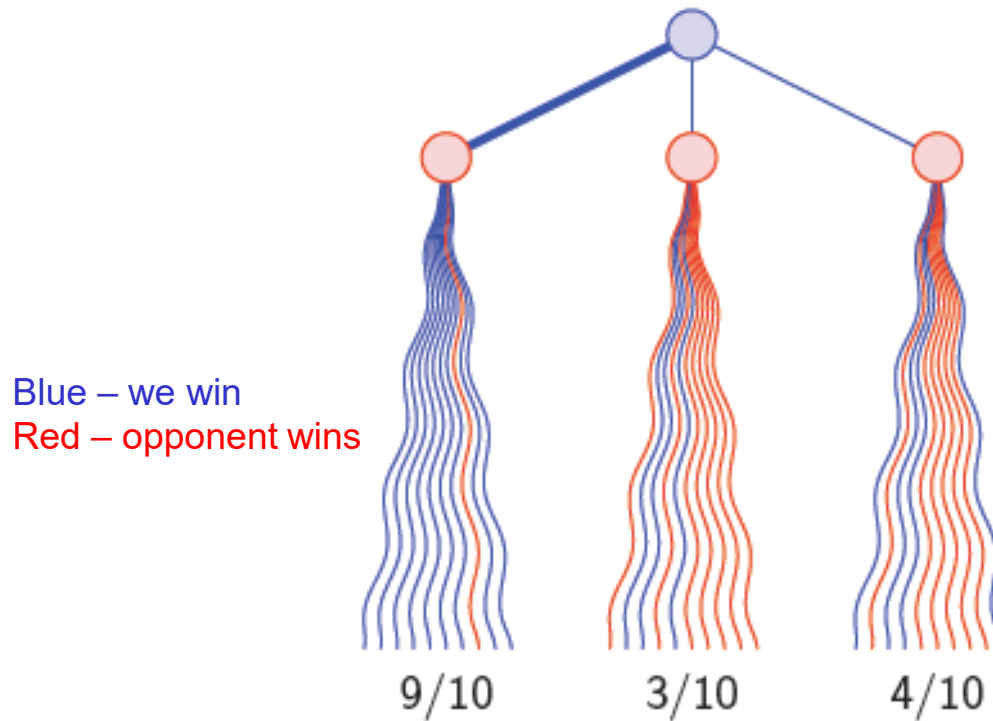
Example

- 3 possible next moves
- Simulate 10 games per move (random move choices)



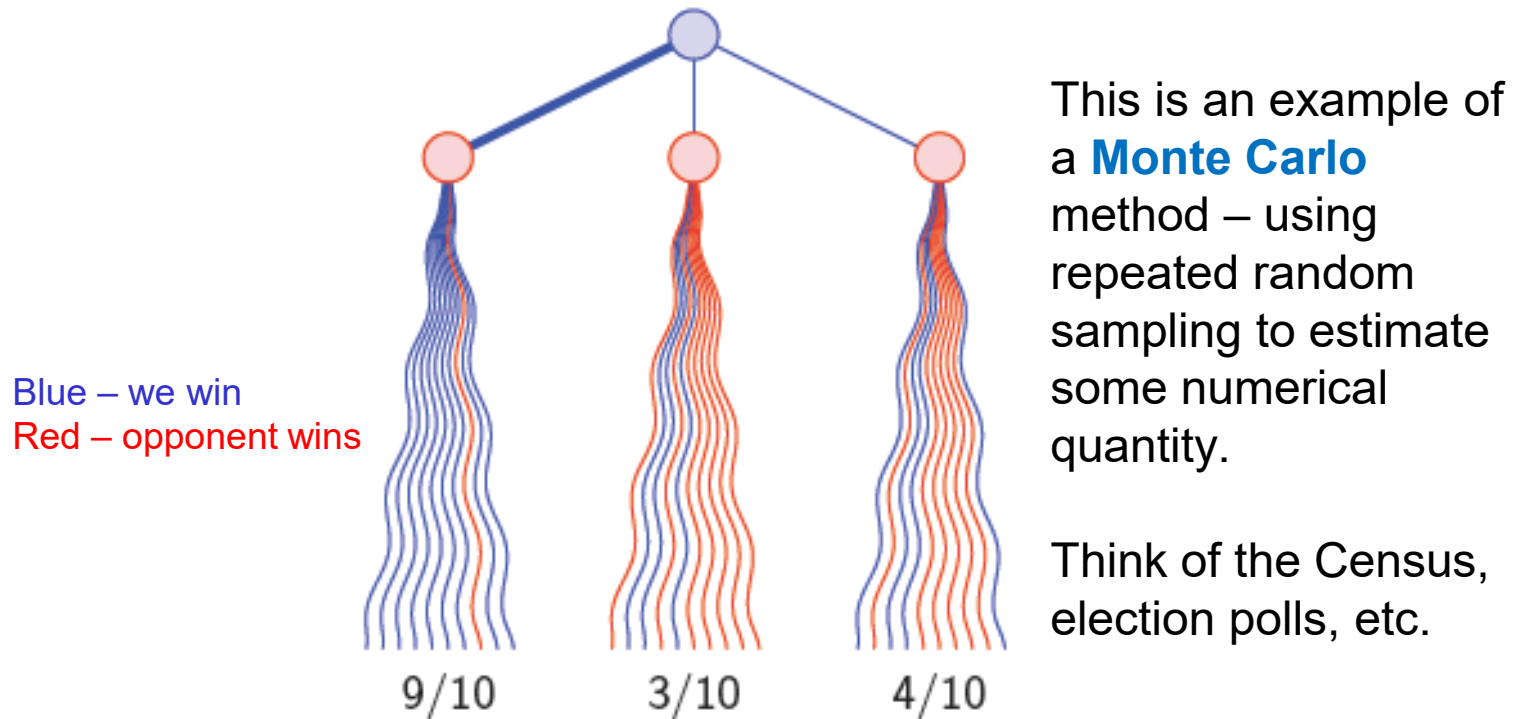
Example

- 3 possible next moves
- Simulate 10 games per move (random move choices)



Example

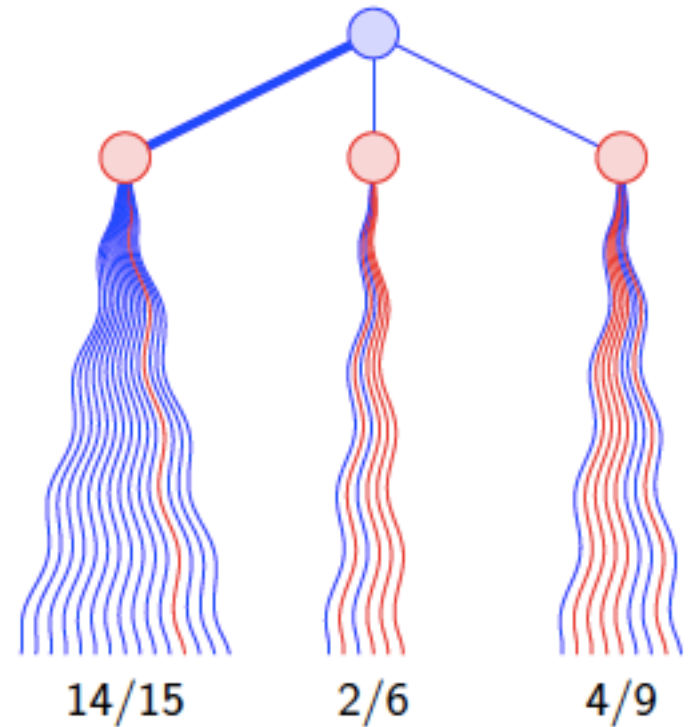
- 3 possible next moves
- Simulate 10 games per move (random move choices)



→ Pick the first move since it has the highest win rate

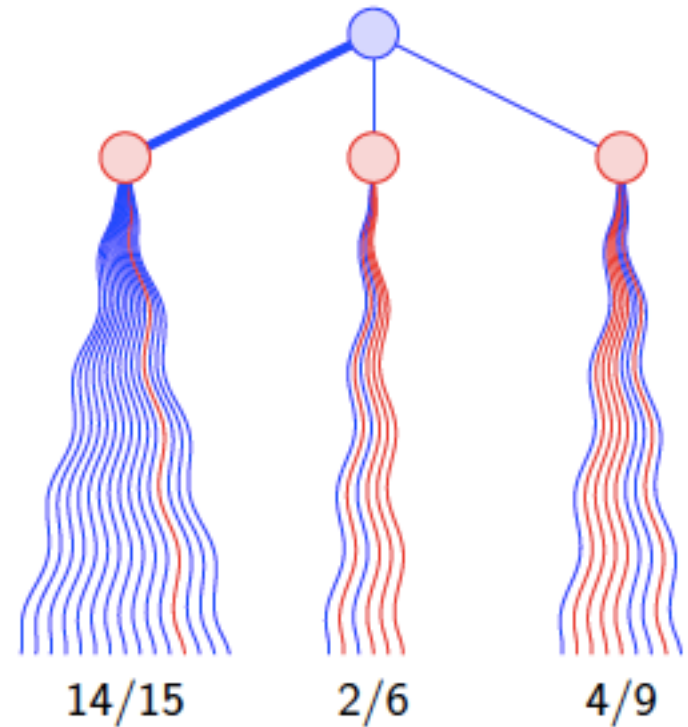
Where to spend the search effort?

- Simulated lines of play do not have to be allocated equally to every move



Where to spend the search effort?

- Simulated lines of play do not have to be allocated equally to every move
- **Intuition:** look more closely at the promising moves, since the others wouldn't be picked (α - β logic)



Monte Carlo Tree Search (MCTS)

- Combine search tree expansion with **Monte Carlo simulations!**

- MCTS uses the concept of a **policy**

- A policy is a mapping, $\pi : S \rightarrow A$, for all states

$$\pi(s) = a$$

- π may select actions deterministically or stochastically

- We'll distinguish two policies:

- **Tree policy** – high-quality, expensive, used near root
- **Default policy** – cheap, used near leaves; e.g., random move

Monte Carlo Tree Search (MCTS)

1. Selection

- Use a **tree policy** on nodes seen before to select a promising path in the search tree (balance best play vs exploration – more to come on this later)

2. Expansion

- Expand the tree when we reach the frontier (leaf nodes)

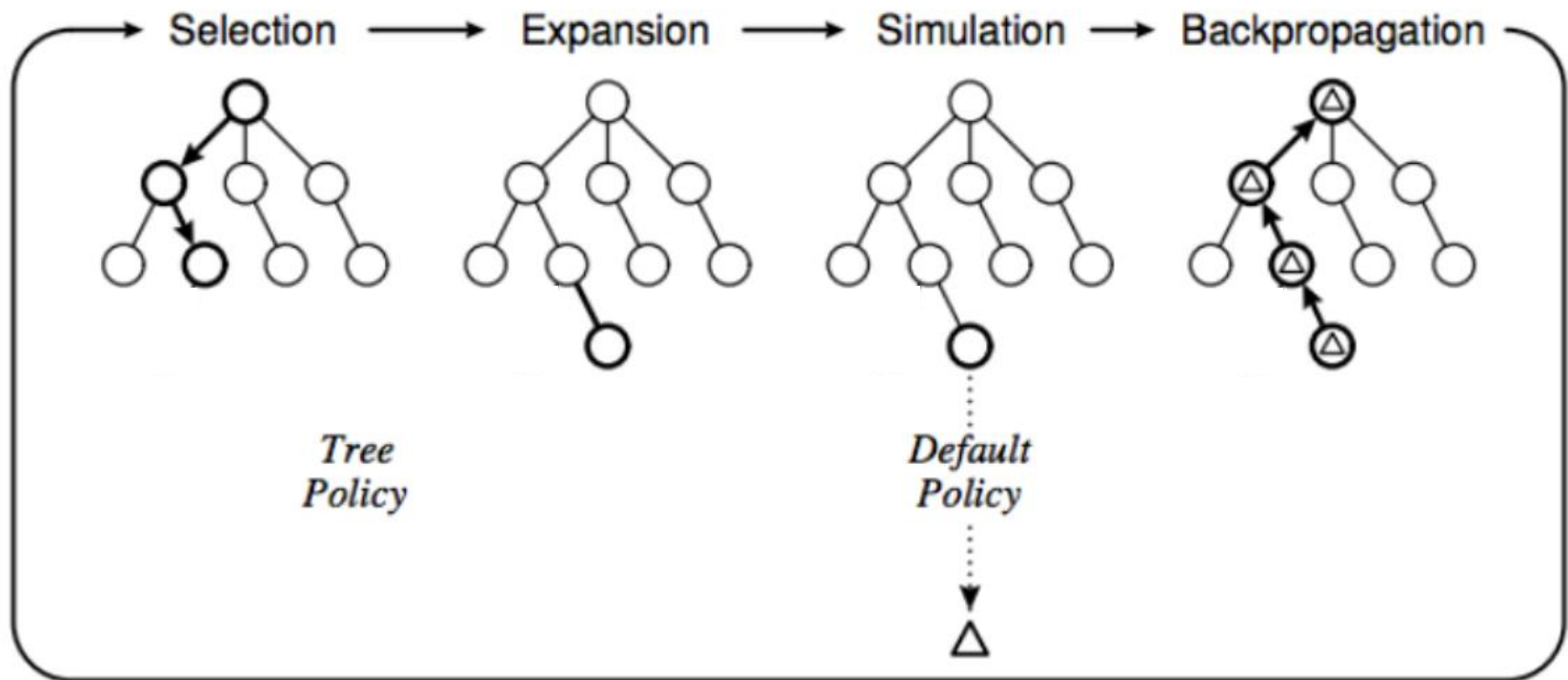
3. Simulation

- From an expansion node, sample playouts (**rollouts**) using a **default policy** for both players (possibly completely random)

4. Backpropagation (like back ups)

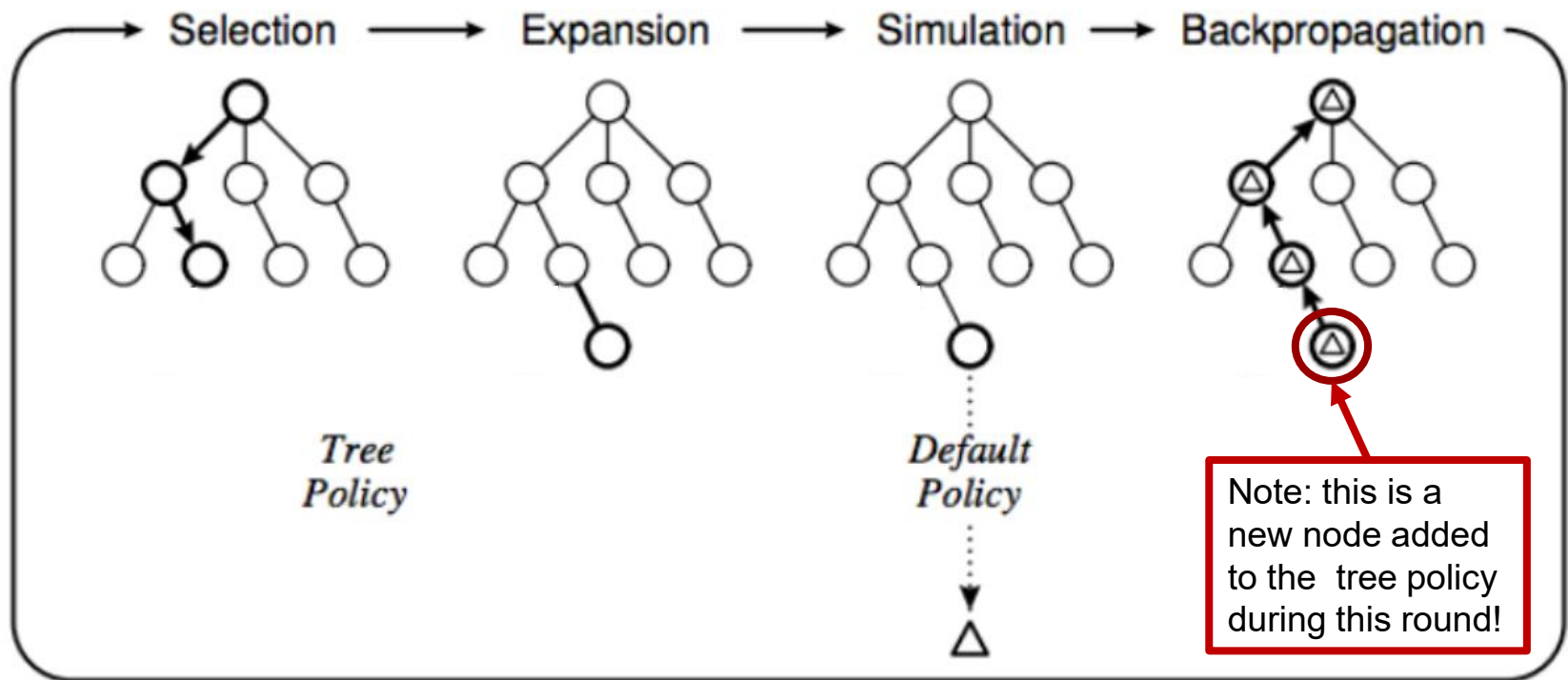
- After the rollout reaches a terminal node, update **value** and visit counts for states visited during selection and expansion

MCTS Steps



adapted from Couetoux et al. (2013)

MCTS Steps



adapted from Couetoux et al. (2013)

MCTS step by step

Simulation 1

Tree Policy



new node in the tree



node stored in tree



state visited but not stored



terminal state



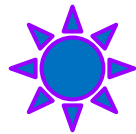
current simulation



previous simulation

MCTS step by step

Simulation 1



Tree Policy



new node in the tree



node stored in tree



state visited but not stored



terminal state



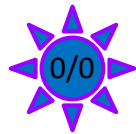
current simulation



previous simulation

MCTS step by step

Simulation 1



Tree Policy



new node in the tree



node stored in tree



state visited but not stored



terminal state



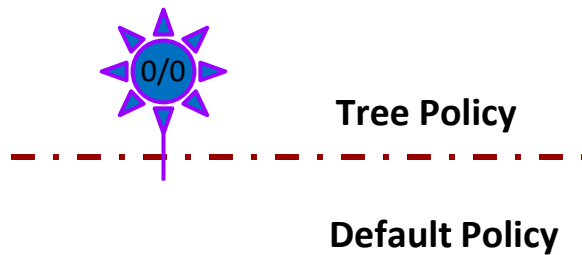
current simulation



previous simulation

MCTS step by step

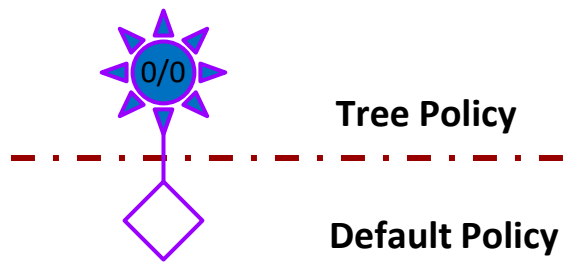
Simulation 1



-  new node in the tree
-  node stored in tree
-  state visited but not stored
-  terminal state
-  current simulation
-  previous simulation

MCTS step by step

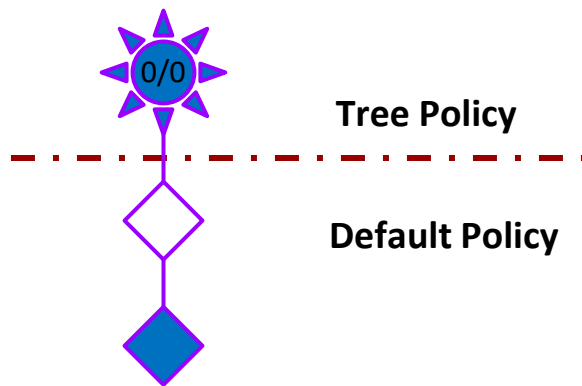
Simulation 1



-  new node in the tree
-  node stored in tree
-  state visited but not stored
-  terminal state
-  current simulation
-  previous simulation

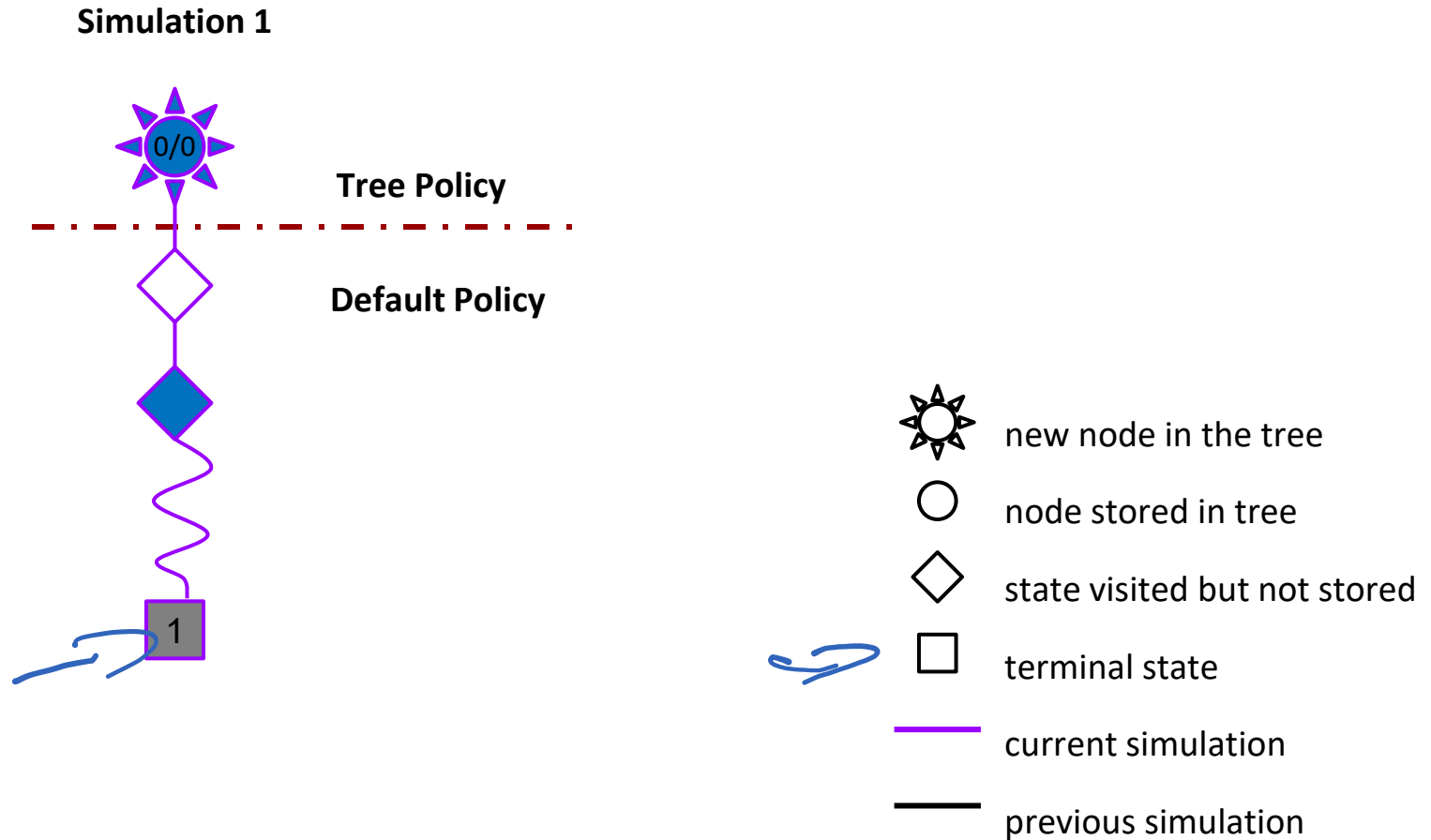
MCTS step by step

Simulation 1

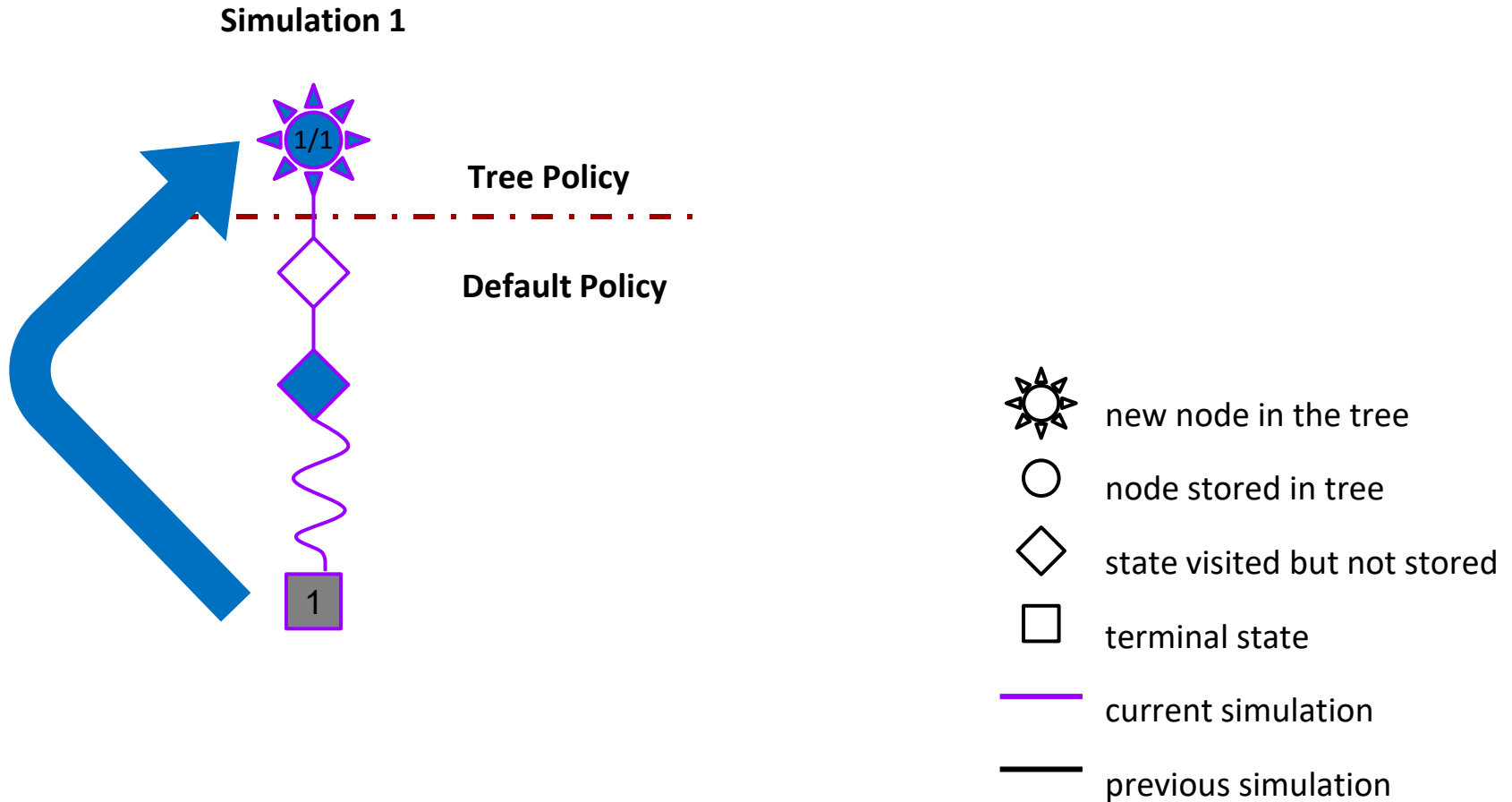


-  new node in the tree
-  node stored in tree
-  state visited but not stored
-  terminal state
-  current simulation
-  previous simulation

MCTS step by step



MCTS step by step



MCTS step by step

Simulation 2



Tree Policy



new node in the tree



node stored in tree



state visited but not stored



terminal state



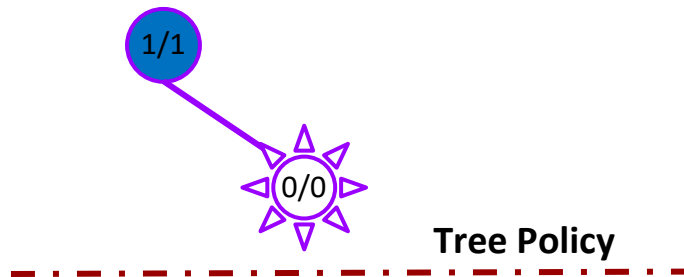
current simulation



previous simulation

MCTS step by step

Simulation 2



new node in the tree



node stored in tree



state visited but not stored



terminal state



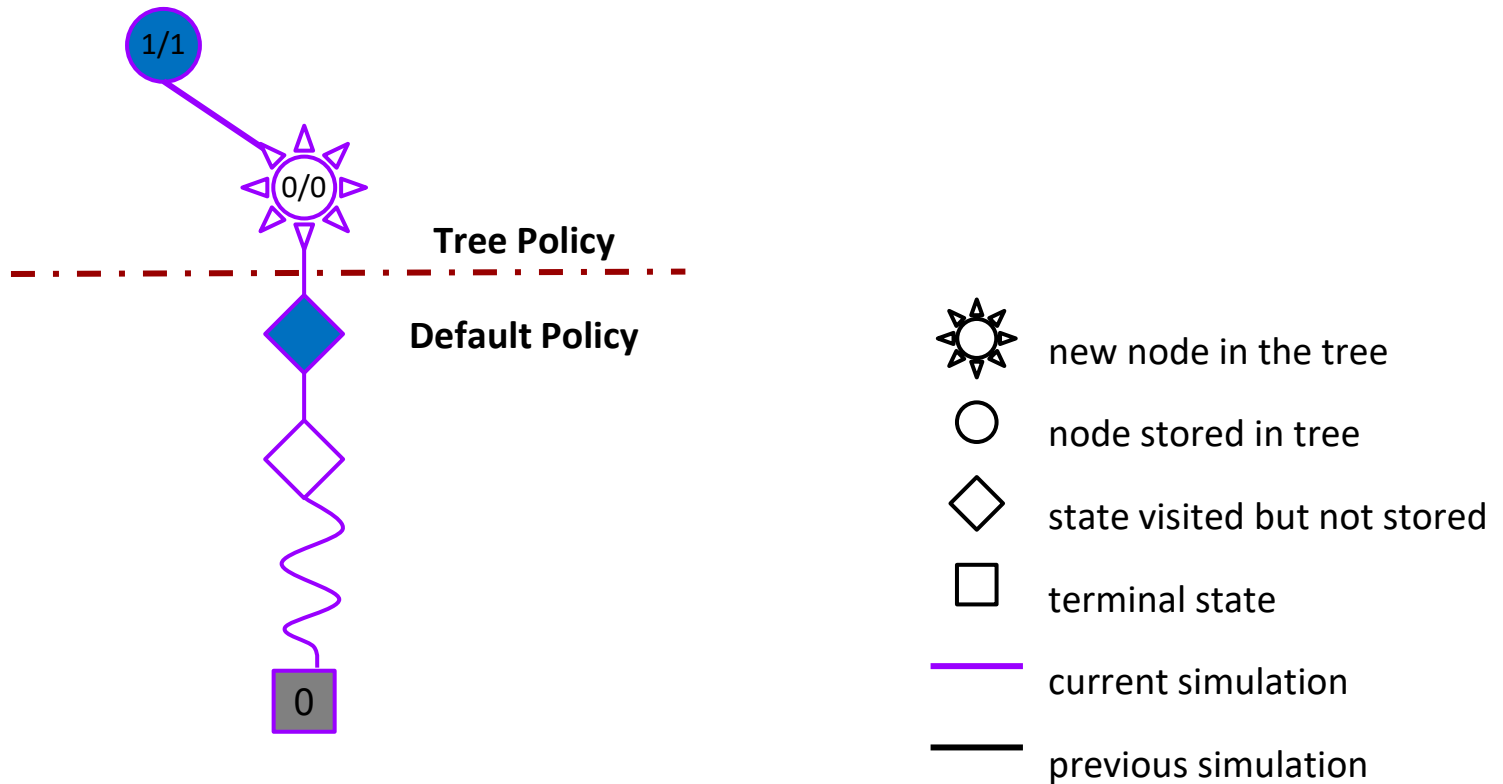
current simulation



previous simulation

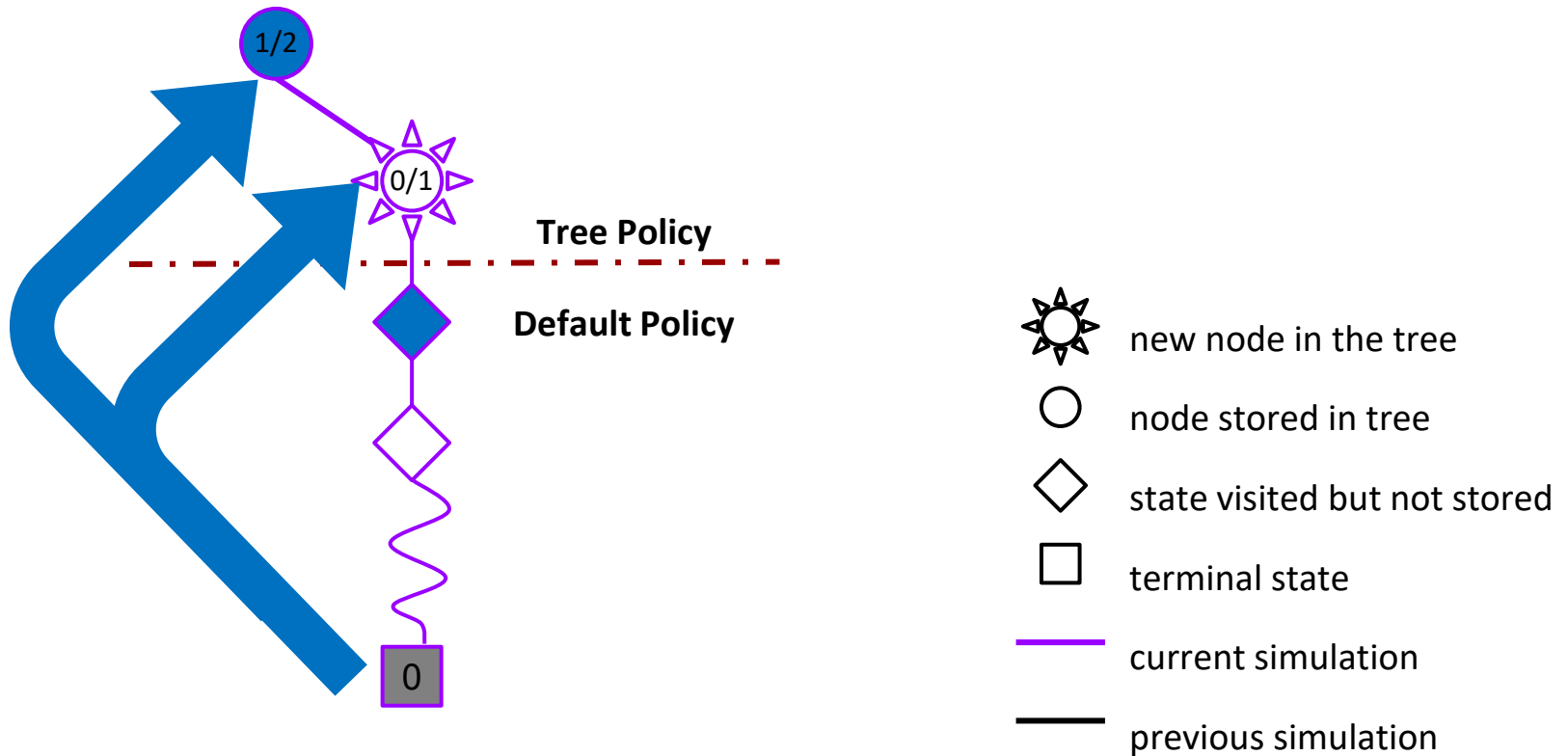
MCTS step by step

Simulation 2



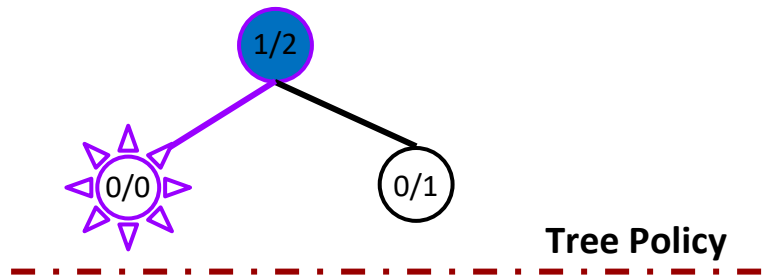
MCTS step by step

Simulation 2



MCTS step by step

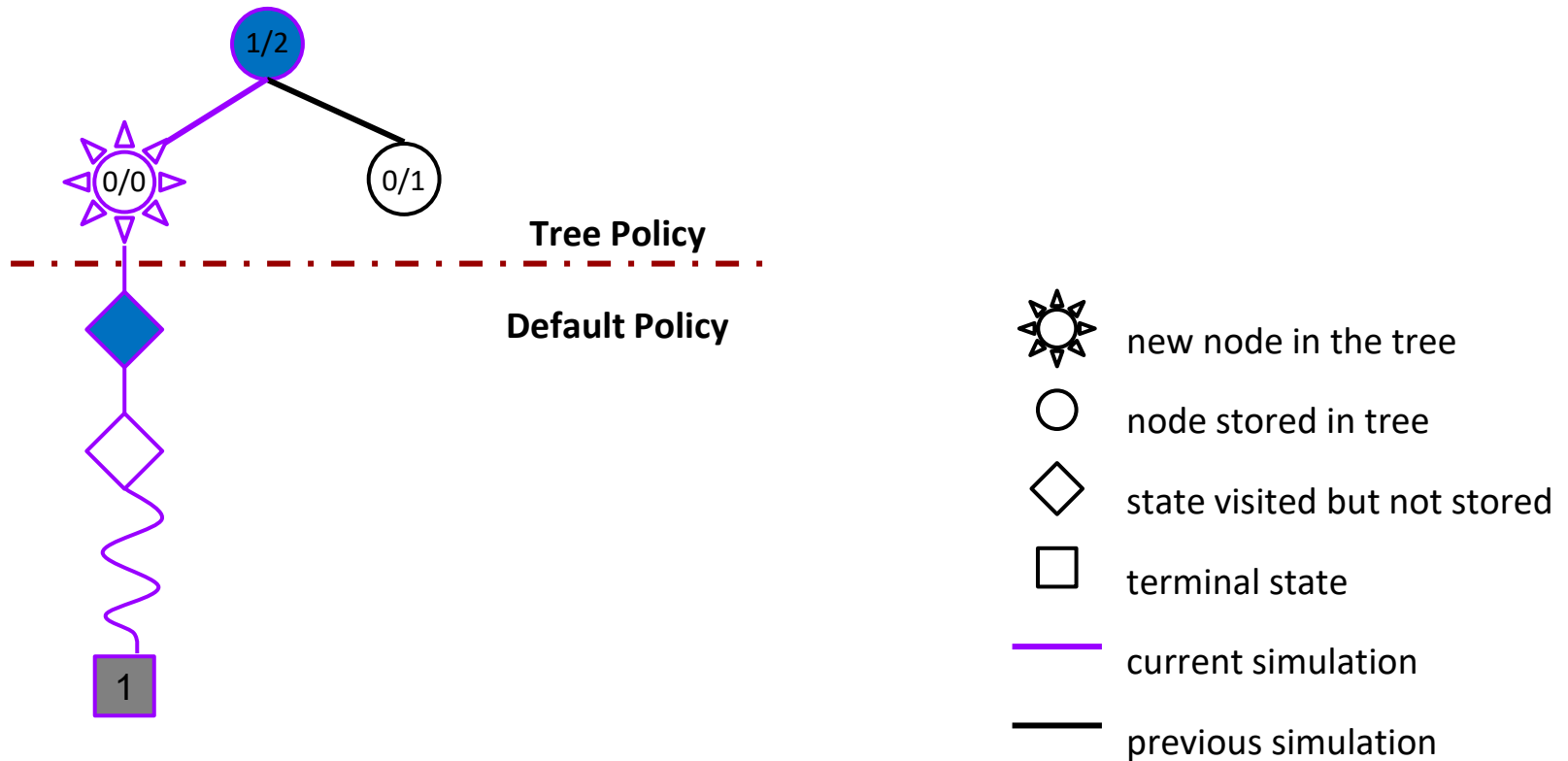
Simulation 3



-  new node in the tree
-  node stored in tree
-  state visited but not stored
-  terminal state
-  current simulation
-  previous simulation

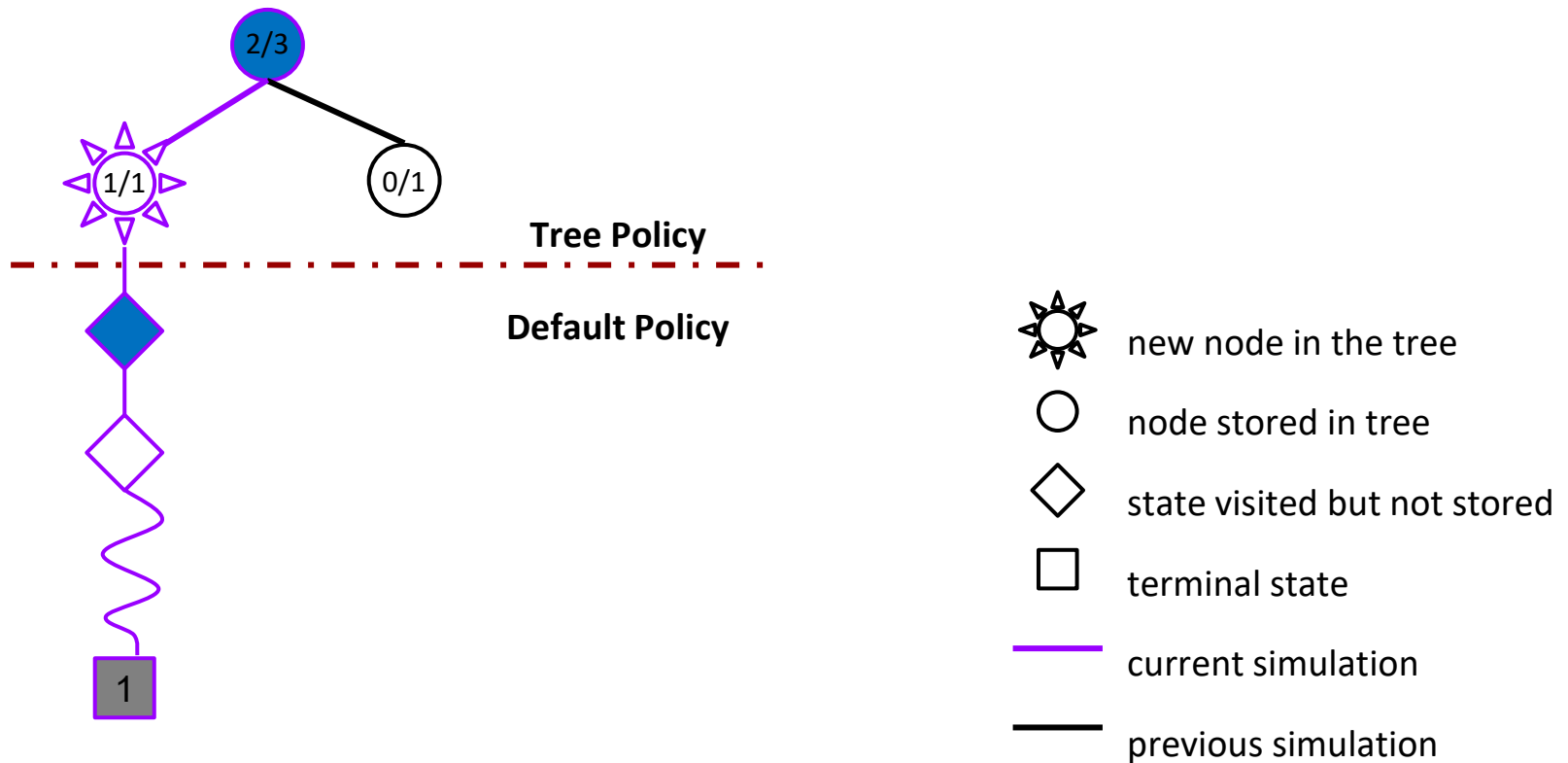
MCTS step by step

Simulation 3



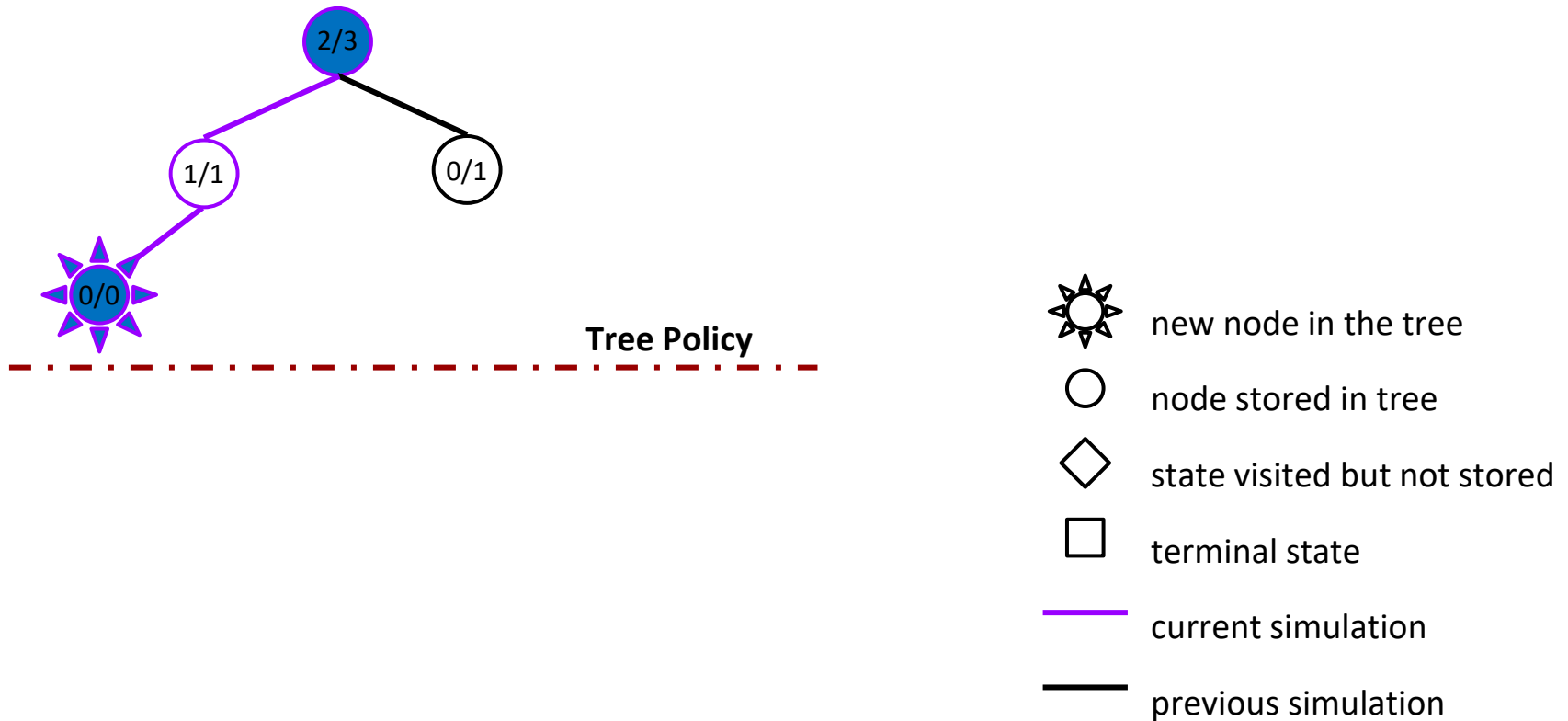
MCTS step by step

Simulation 3



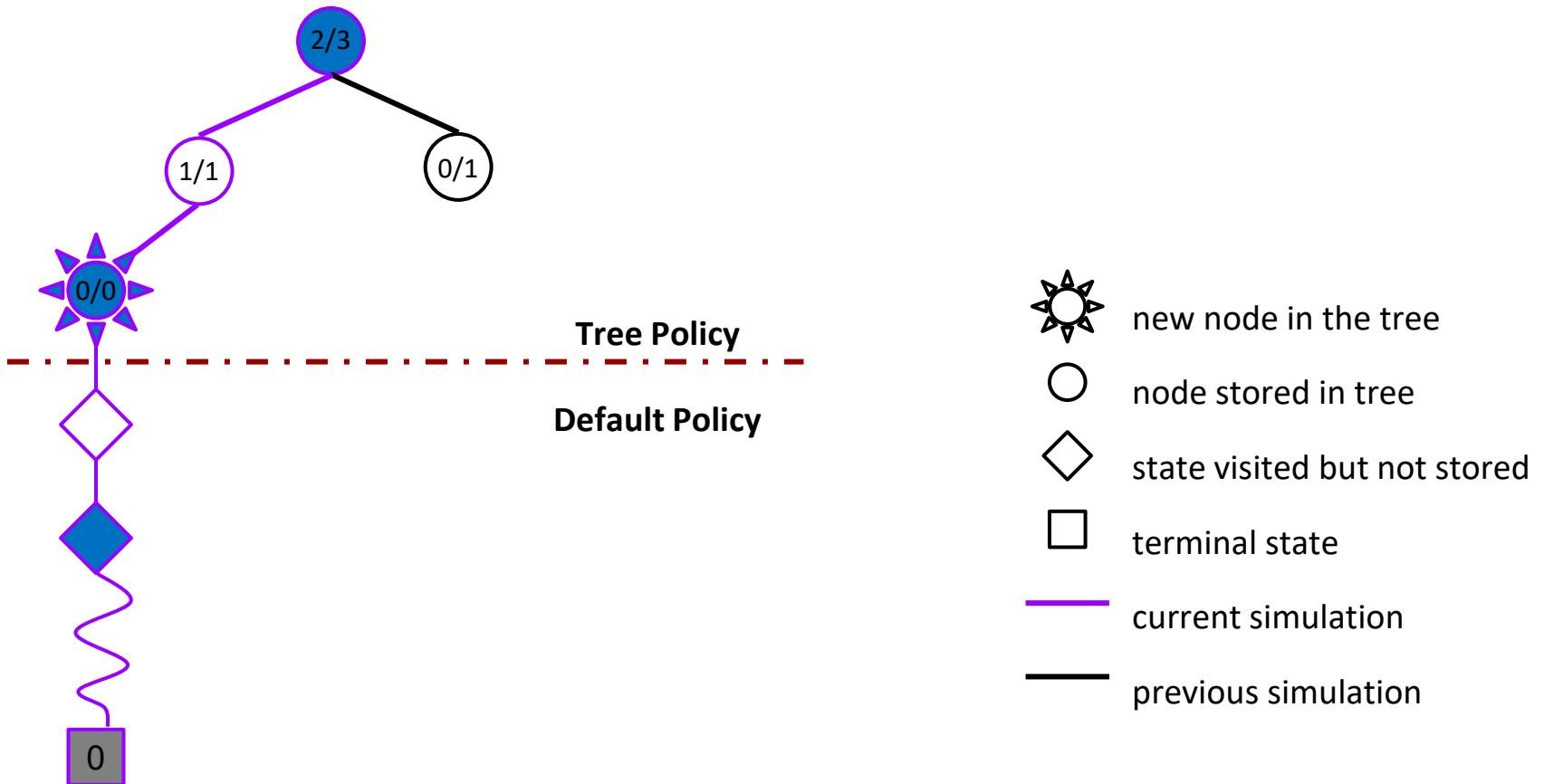
MCTS step by step

Simulation 4



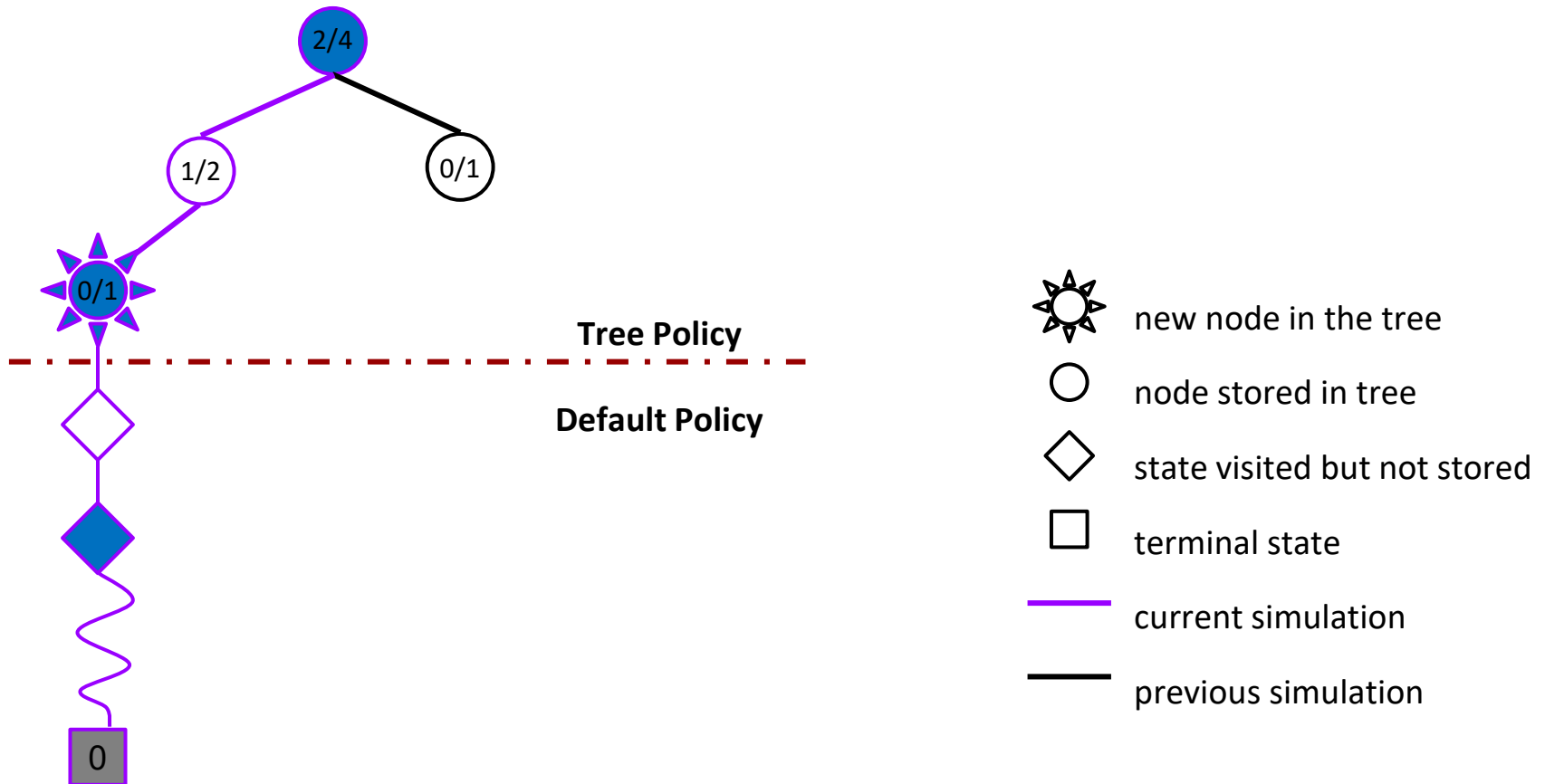
MCTS step by step

Simulation 4



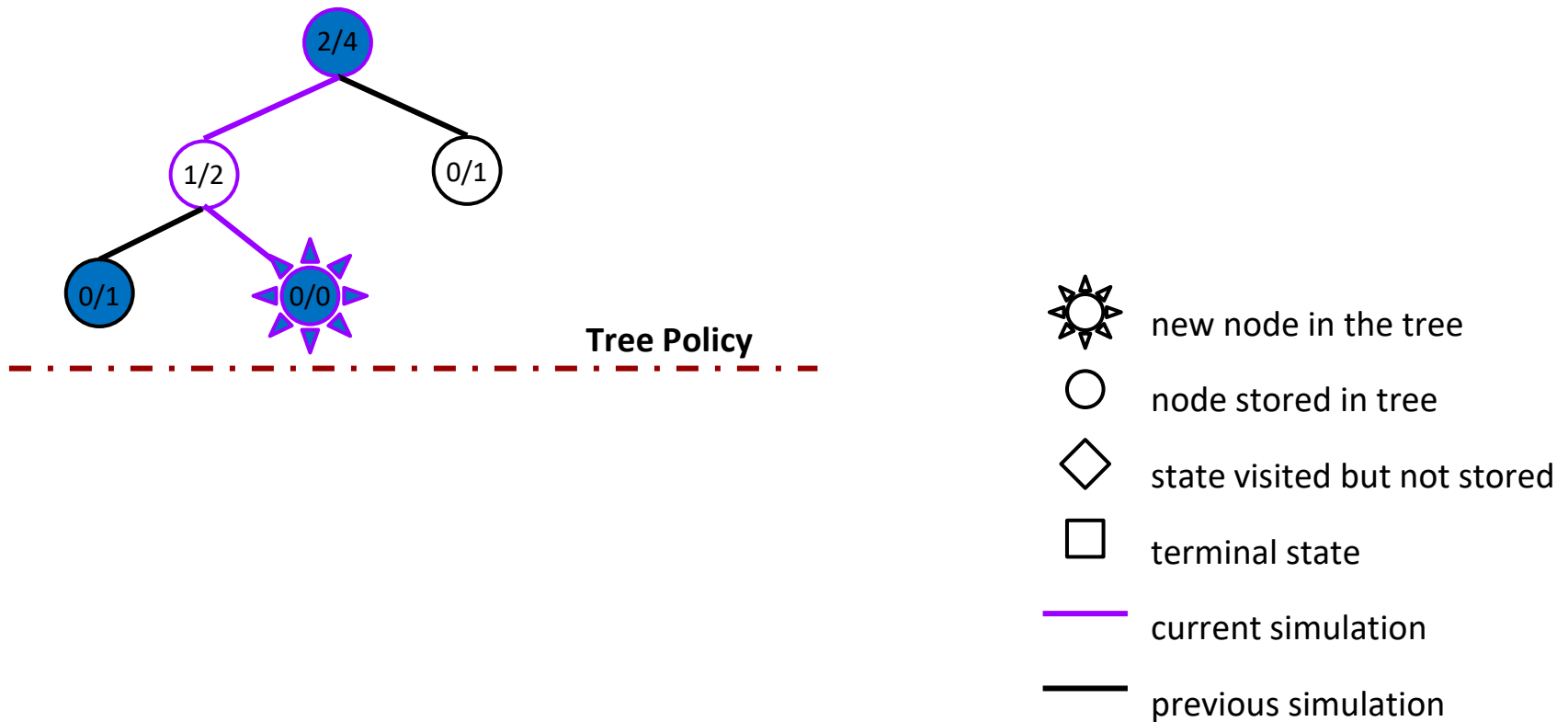
MCTS step by step

Simulation 4



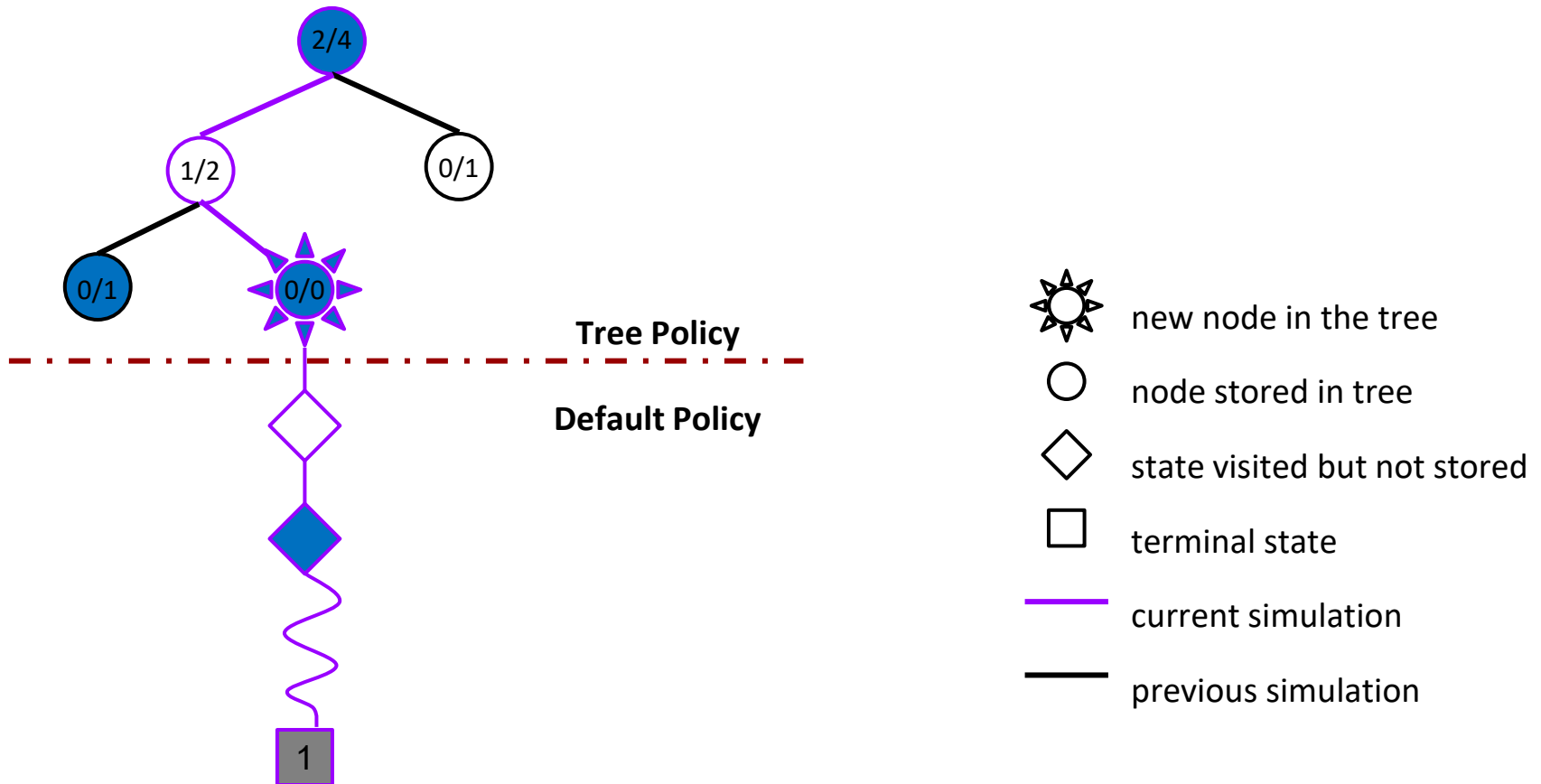
MCTS step by step

Simulation 5



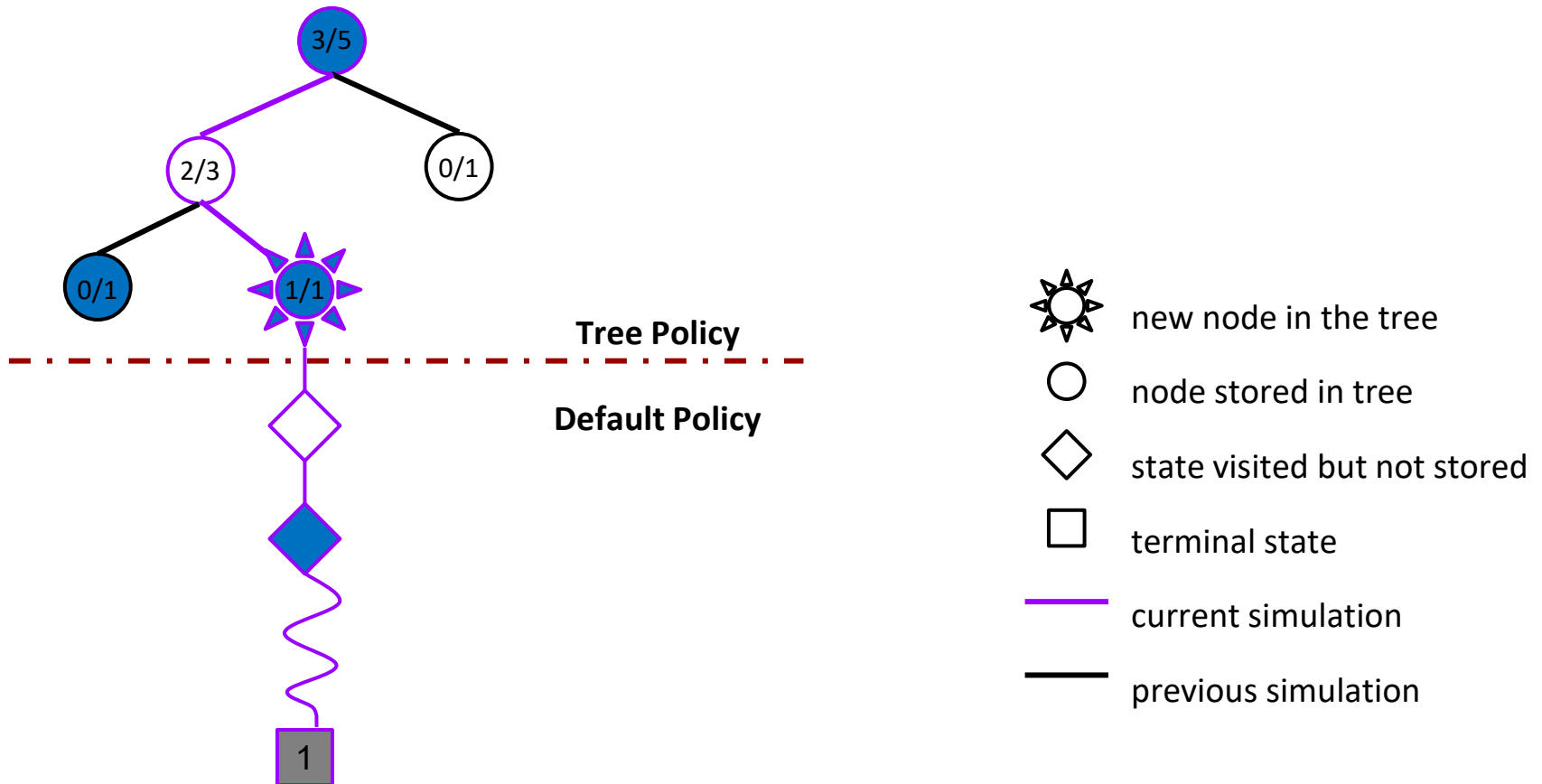
MCTS step by step

Simulation 5

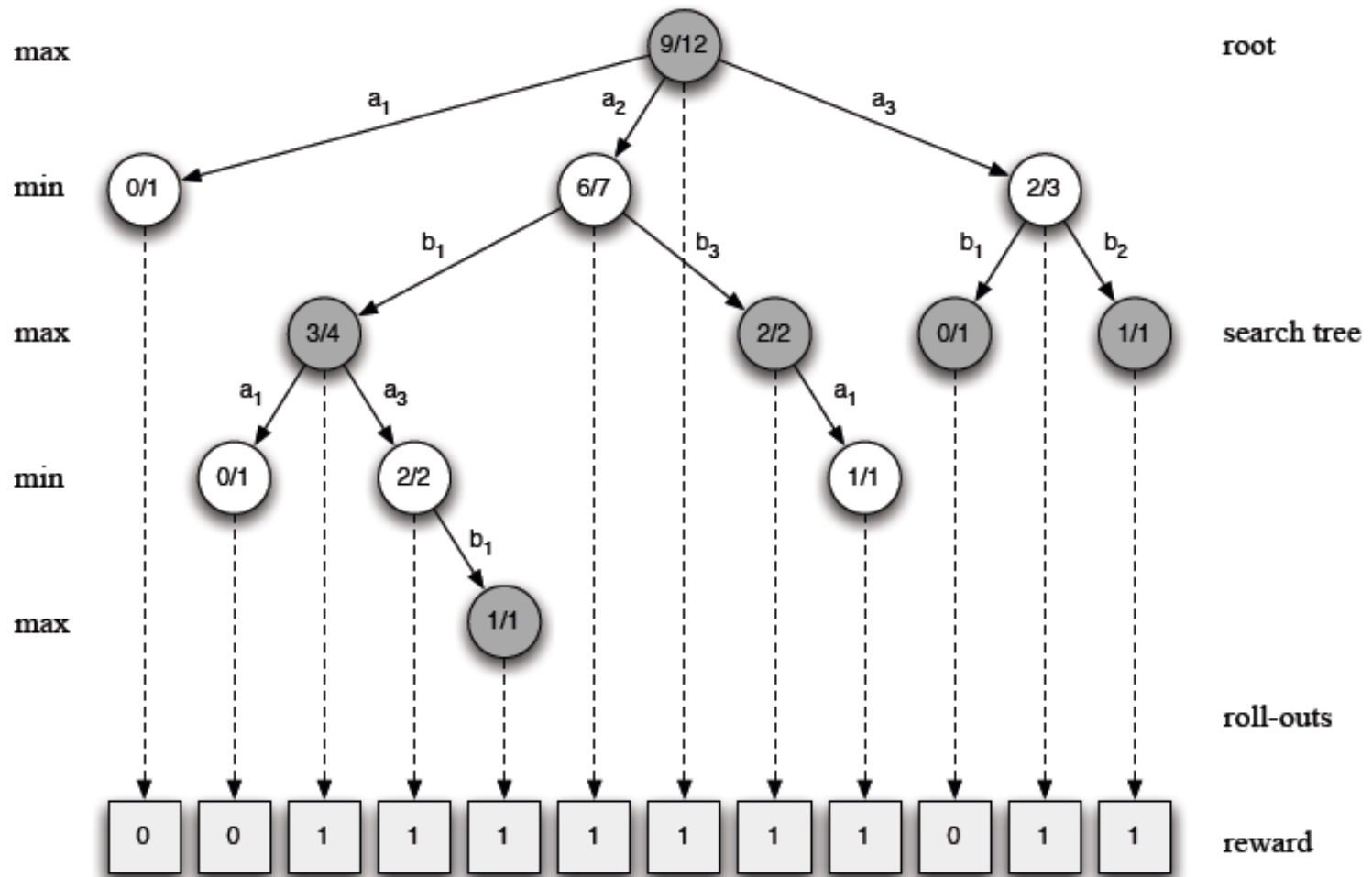


MCTS step by step

Simulation 5



Example of Accumulating Statistics



MCTS Properties

- The purely MC evaluation function depends only on the observed outcomes of simulations
 - No hand-crafted features, weights, or functions
- The evaluation function continues to improve from additional simulations
 - In the limit of infinite memory and computation, it converges on the optimal search tree (i.e., **minimax**)
- The search develops in a selective, best-first manner; It expands promising regions of the search space much more deeply

Tree Policy

- How should we select the next moves in the search tree?
- We need to balance two opposing concerns:
 - **Exploitation**: pick a node that looks promising according to current estimates based on previous simulations
 - **Exploration**: pick a node that hasn't participated in many simulations yet; we want more information on it

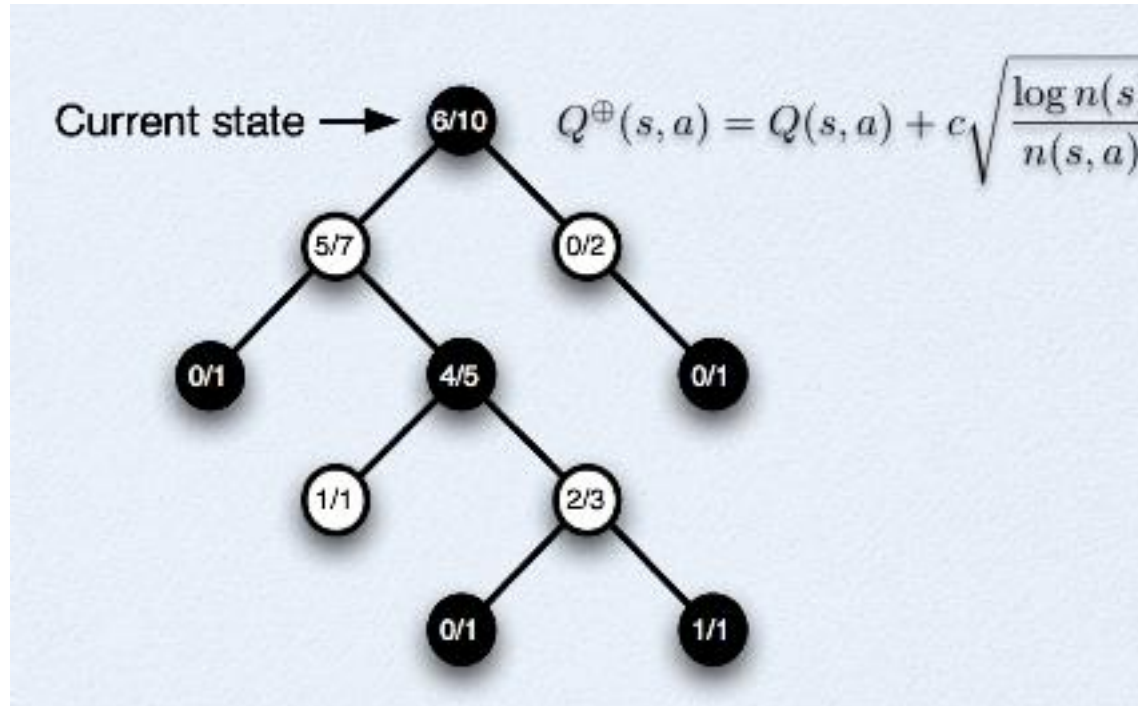
Tree Policy

- How should we select the next moves in the search tree?
- We need to balance two opposing concerns:
 - **Exploitation**: pick a node that looks promising according to current estimates based on previous simulations
 - **Exploration**: pick a node that hasn't participated in many simulations yet; we want more information on it

Let's define the following:

- $Q(s, a) \equiv$ the value of action a in state s , based on simulations so far
 - Note that max will value positive utility, min will value negative
- $n(s, a) \equiv$ the number of times we have taken action a in state s
- $n(s) \equiv$ the number of times we have visited s in simulations

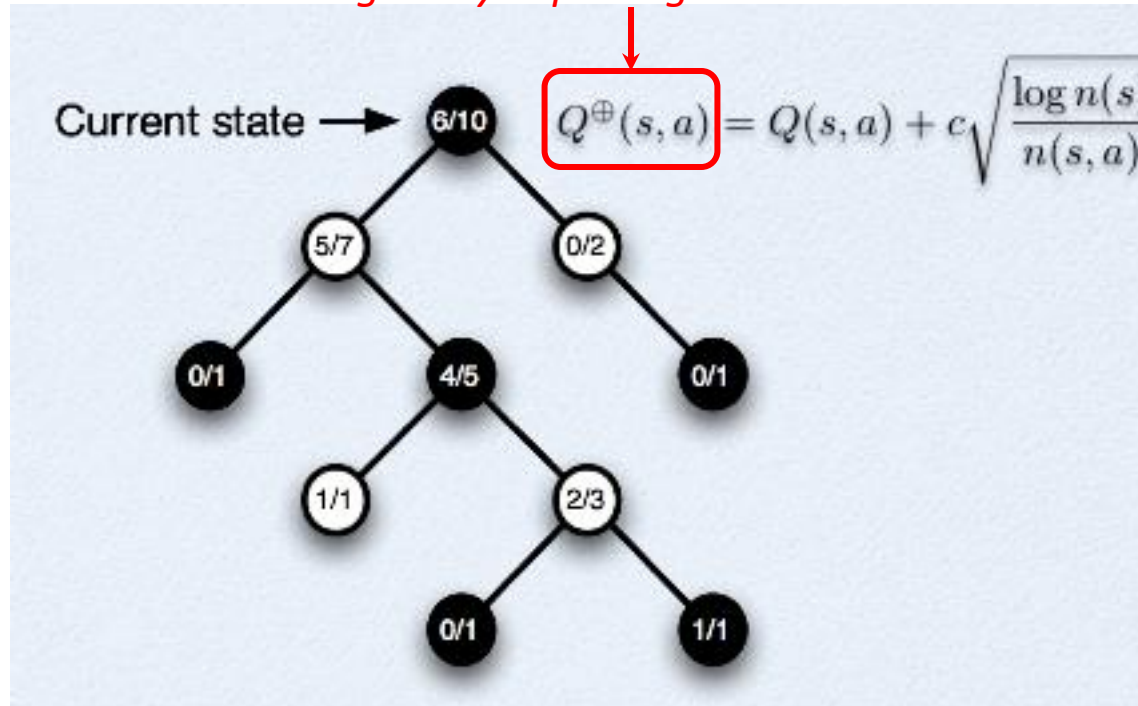
Upper Confidence Trees (UCT)



- $Q(s, a)$ value of taking action a in state s
- $n(s, a)$ number of times we have taken action a in state s
- $n(s)$ number of times we have visited s in simulations
- c scaling constant

Upper Confidence Trees (UCT)

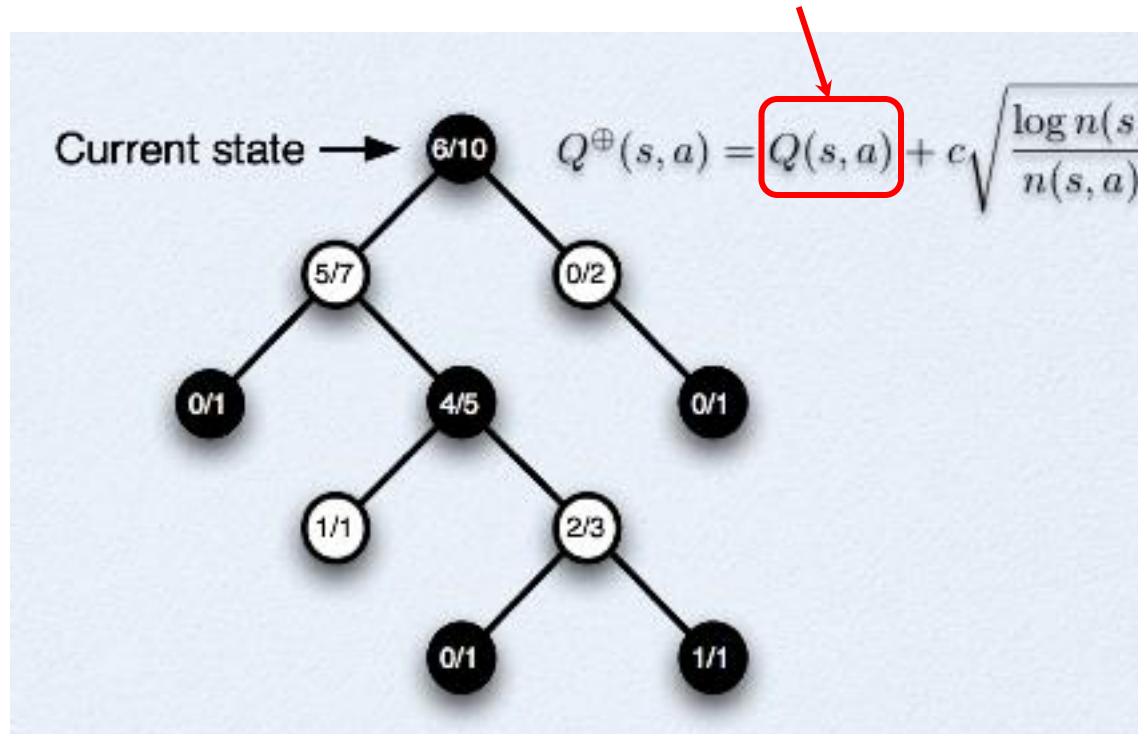
Approximates the value of taking action a in state s , by how much we might still gain by exploring more.



- $Q(s, a)$ value of taking action a in state s
- $n(s, a)$ number of times we have taken action a in state s
- $n(s)$ number of times we have visited s in simulations
- c scaling constant

Upper Confidence Trees (UCT)

Exploitation (= current estimate)



$Q(s, a)$ value of taking action a in state s

$n(s, a)$ number of times we have taken action a in state s

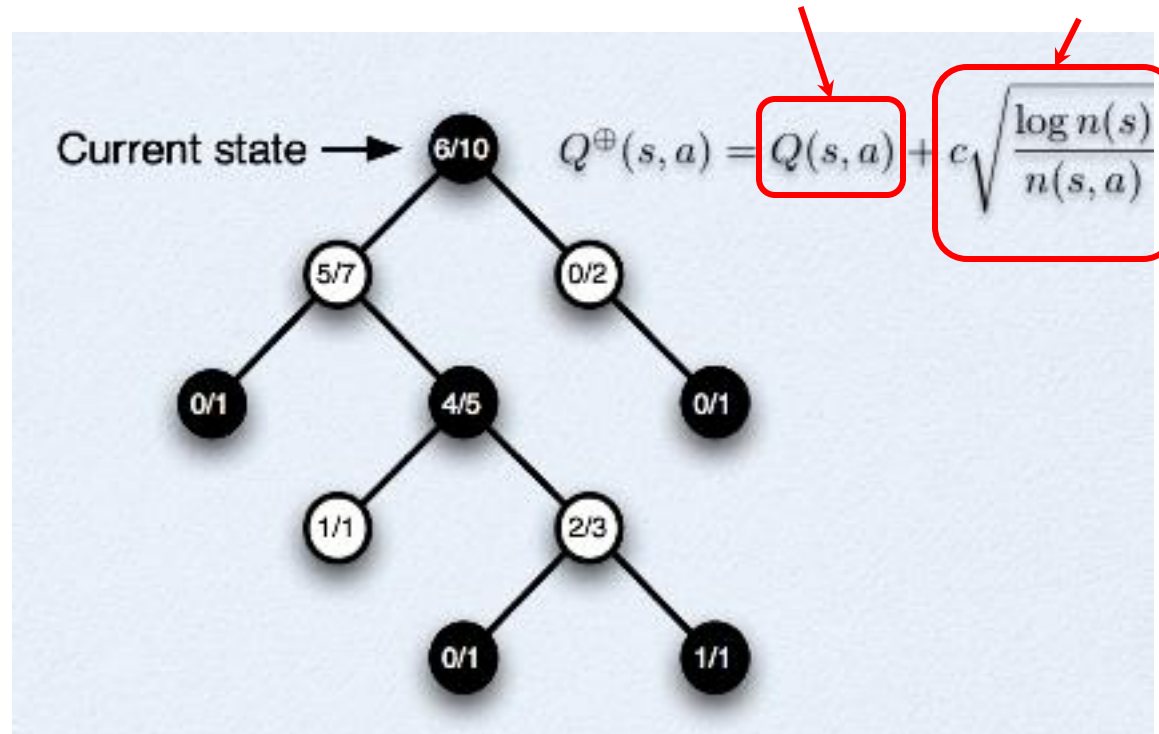
$n(s)$ number of times we have visited s in simulations

c scaling constant

Upper Confidence Trees (UCT)

Exploitation

Exploration (= uncertainty)



$Q(s, a)$ value of taking action a in state s

$n(s, a)$ number of times we have taken action a in state s

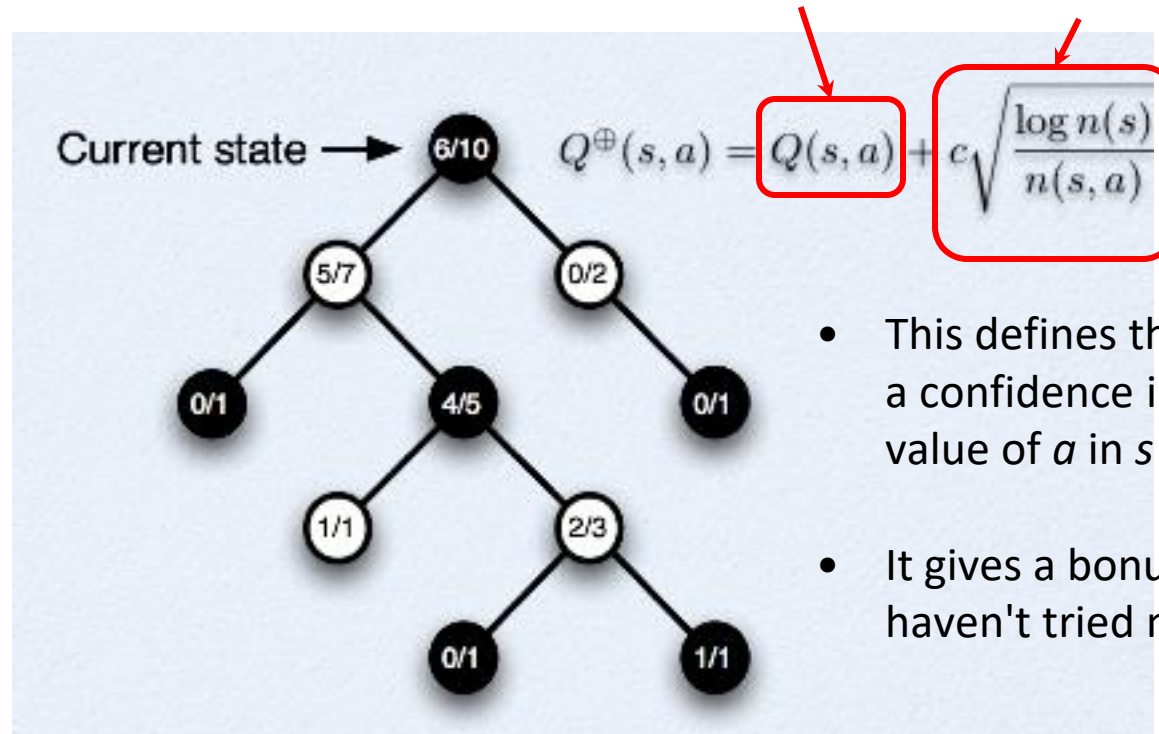
$n(s)$ number of times we have visited s in simulations

c scaling constant

Upper Confidence Trees (UCT)

Exploitation

Exploration (= uncertainty)



$Q(s, a)$ value of taking action a in state s

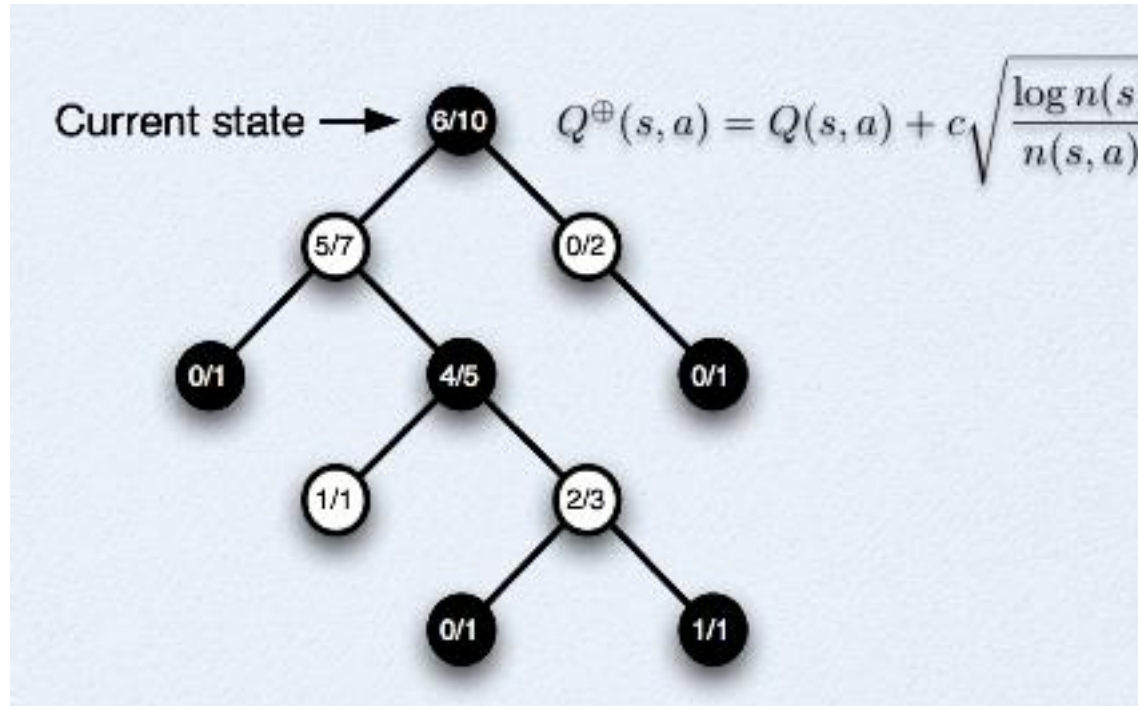
$n(s, a)$ number of times we have taken action a in state s

$n(s)$ number of times we have visited s in simulations

c scaling constant

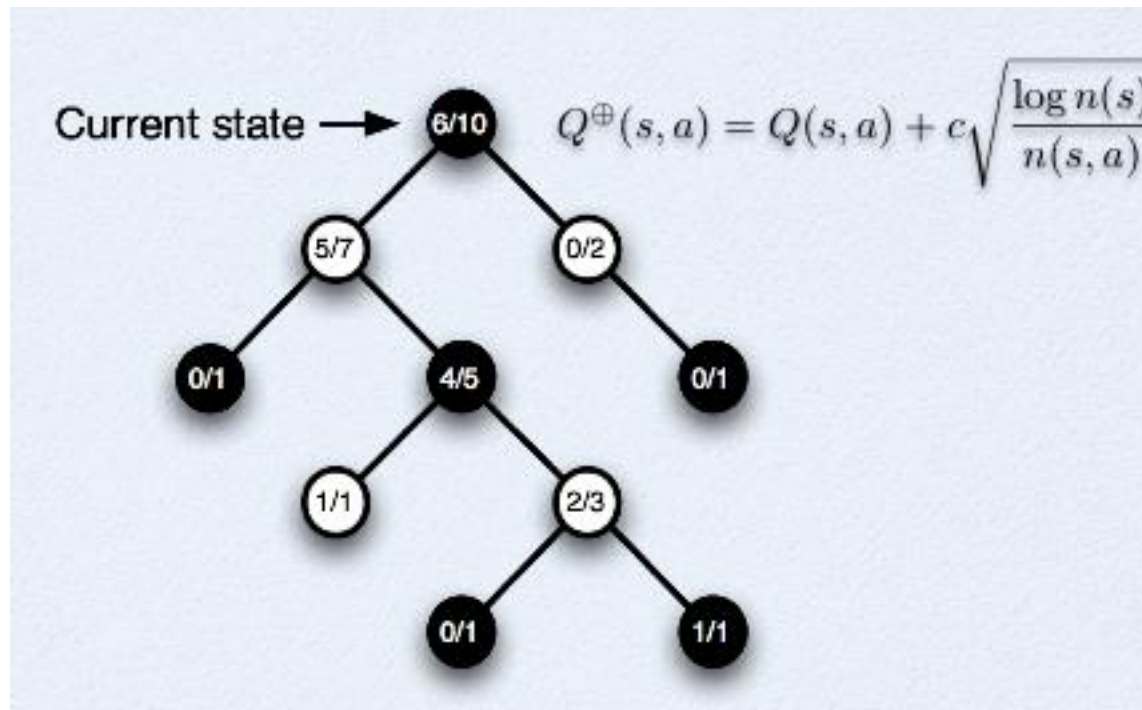
Exercise

Select between the two actions by **black** (max player) at the root of the tree (use $c = \sqrt{2}$, log base e).



- $Q(s, a)$ value of taking action a in state s
- $n(s, a)$ number of times we have taken action a in state s
- $n(s)$ number of times we have visited s in simulations
- c scaling constant

Answer

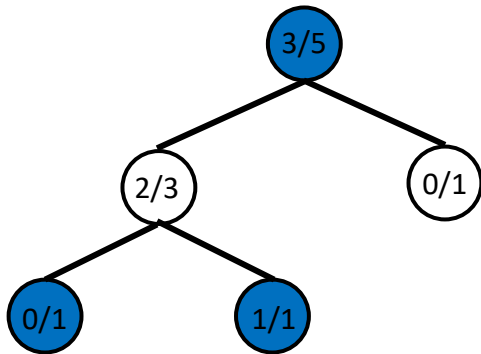


$$Q^*(s, a_1) = \frac{5}{7} + \sqrt{2 \frac{\log 10}{7}} = 1.525$$

$$Q^*(s, a_2) = \frac{0}{2} + \sqrt{2 \frac{\log 10}{2}} = 1.517$$

→ black picks action 1

MCTS questions



1. The computed values at each node are not the sum of the children. Why? Is this an error? Where are the extra values coming from?
2. Why don't we add every single node visited (i.e., visited by the default policy) during every single simulation?
3. Is the tree policy fixed over the entire process or does it change? If so, how?

MCTS in real games

Scrabble

- Stochastic game (letters drawn randomly)
- Imperfect information (can't see opponent's hand)
- Computers have an advantage because they can store and search the dictionary (move generation is easy)



- Quite complex!
 - ~700 branching factor
 - ~25 depth for a game
 - Rough complexity: 10^{70} search states
- Strategy is difficult! What letters to keep?

The Story of Go

- The Chinese game of Go is 2000 years old
- It's considered the hardest classic board game
- It was posed as a grand challenge task for AI
- Traditional approaches to game-tree search are insufficient for victory



The Game of Go

- Game characteristics:
 - $\sim 10^{170}$ unique positions
 - ~ 200 moves long
 - ~ 200 branching factor
 - $\sim 10^{360}$ complexity



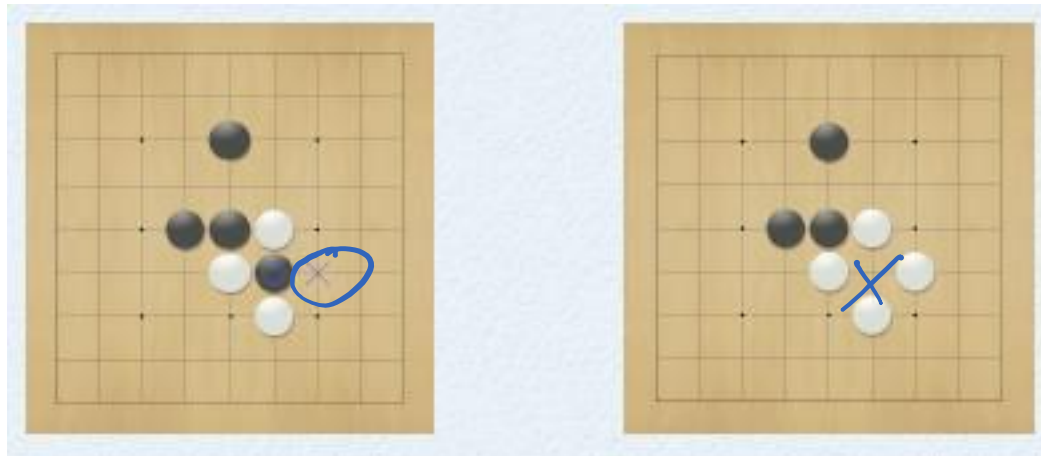
The Rules of Go

- Usually played on 19x19 board (also 13x13 or 9x9 board)
- Simple rules, complex strategy
- Black and white place stones alternately



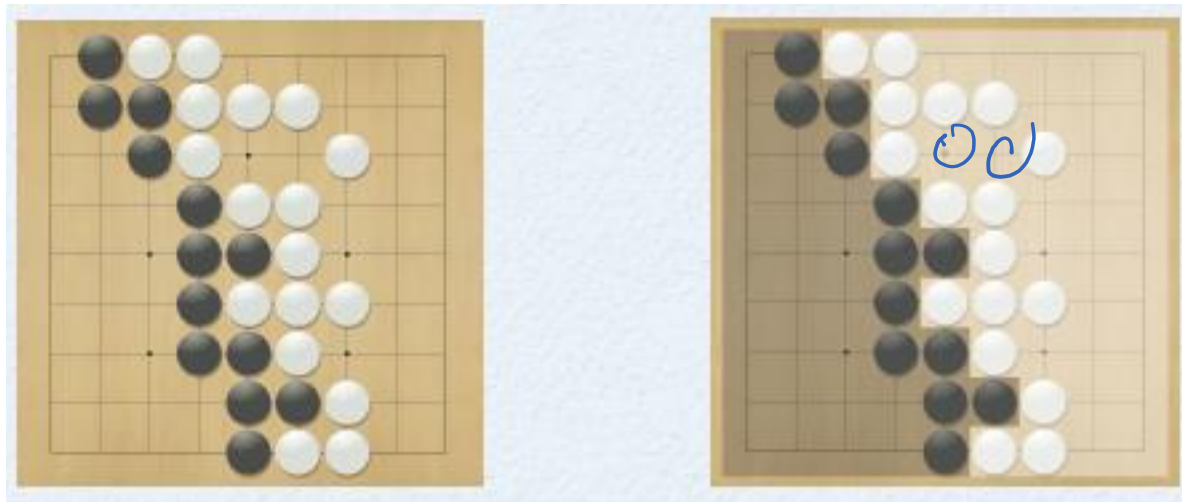
Capturing

- If stones are completely surrounded, they are captured



Winner

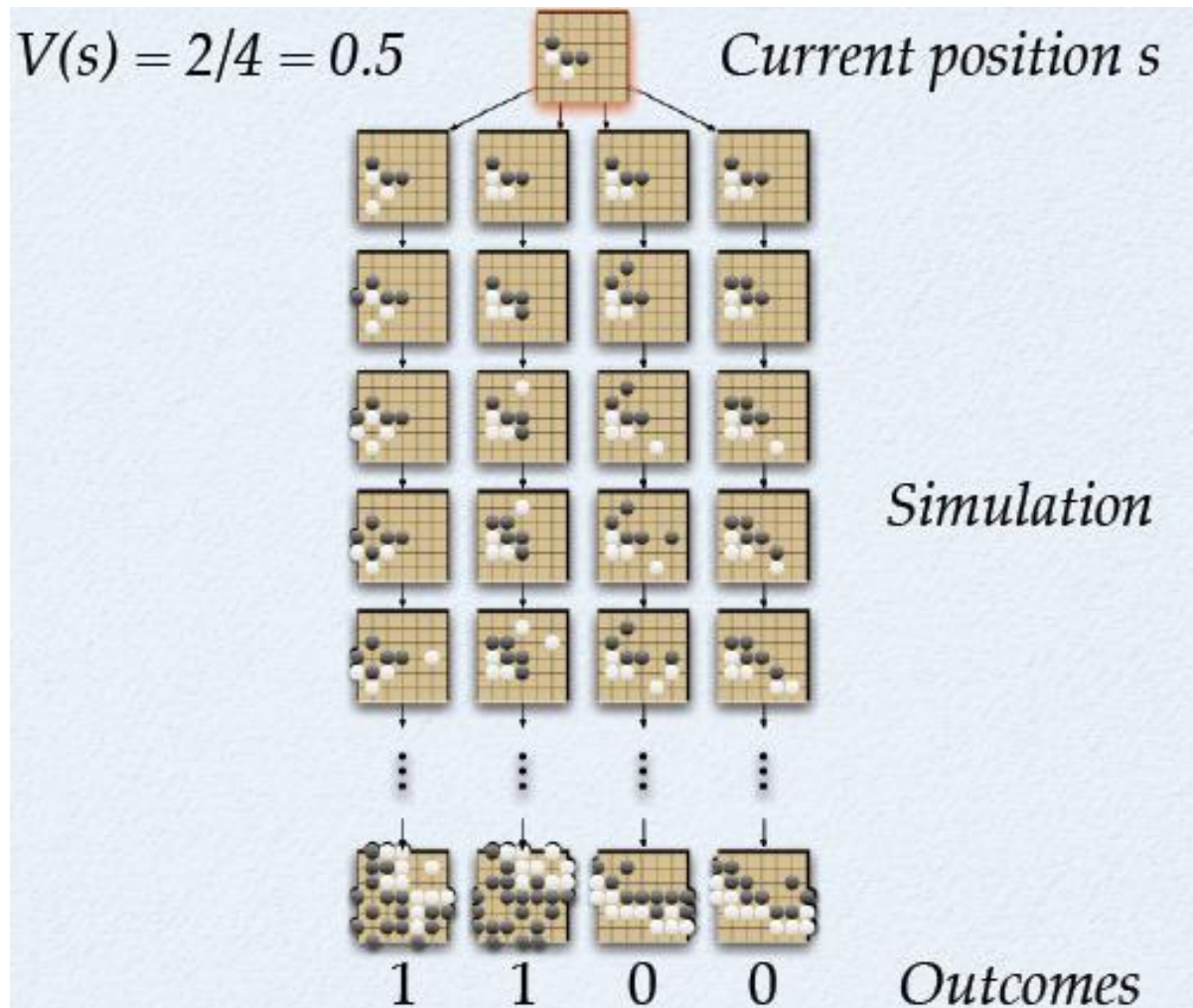
- The game is finished when both players pass
- Intersections surrounded by each player are known as territory
- The player with more territory wins the game



Position Evaluation for AI

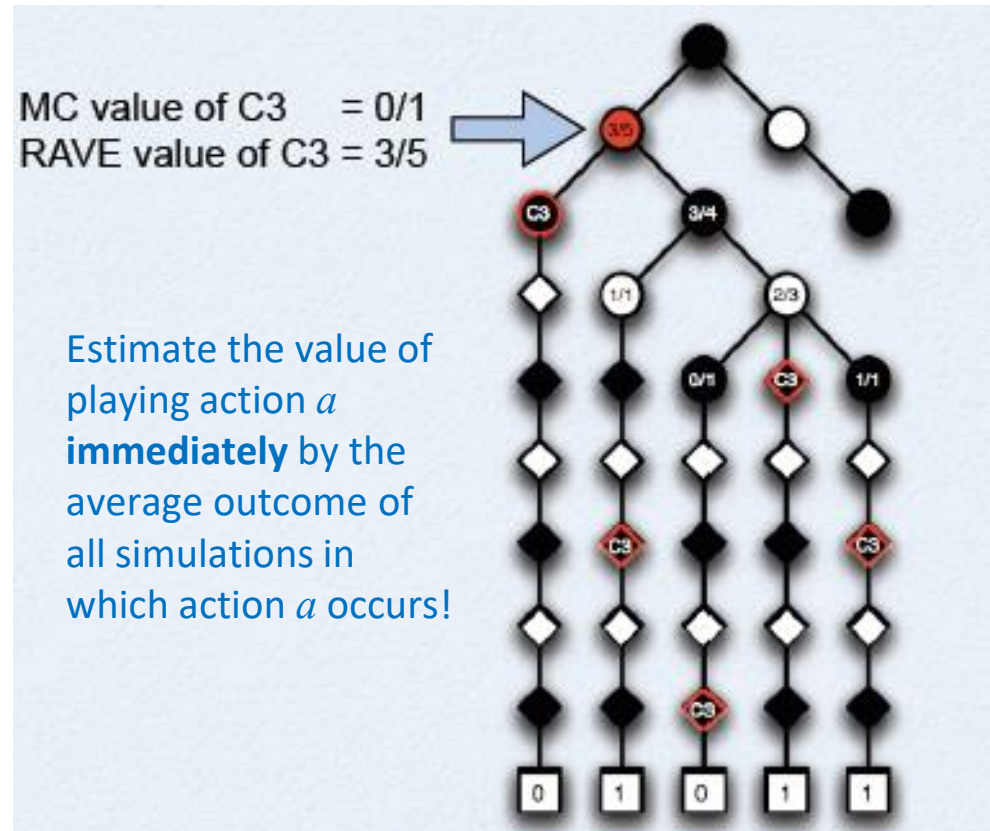
- Game outcome, z
 - Black wins: $z = 1$
 - White wins: $z = 0$
- Value of state s , $V(s)$
 - $V^\pi = E^\pi\{z|s\}$ $\leftarrow$ Monte-Carlo simulation
 - $V^* = \text{minimax}(V^\pi(s))$ $\leftarrow$ Tree search

Monte-Carlo Simulation



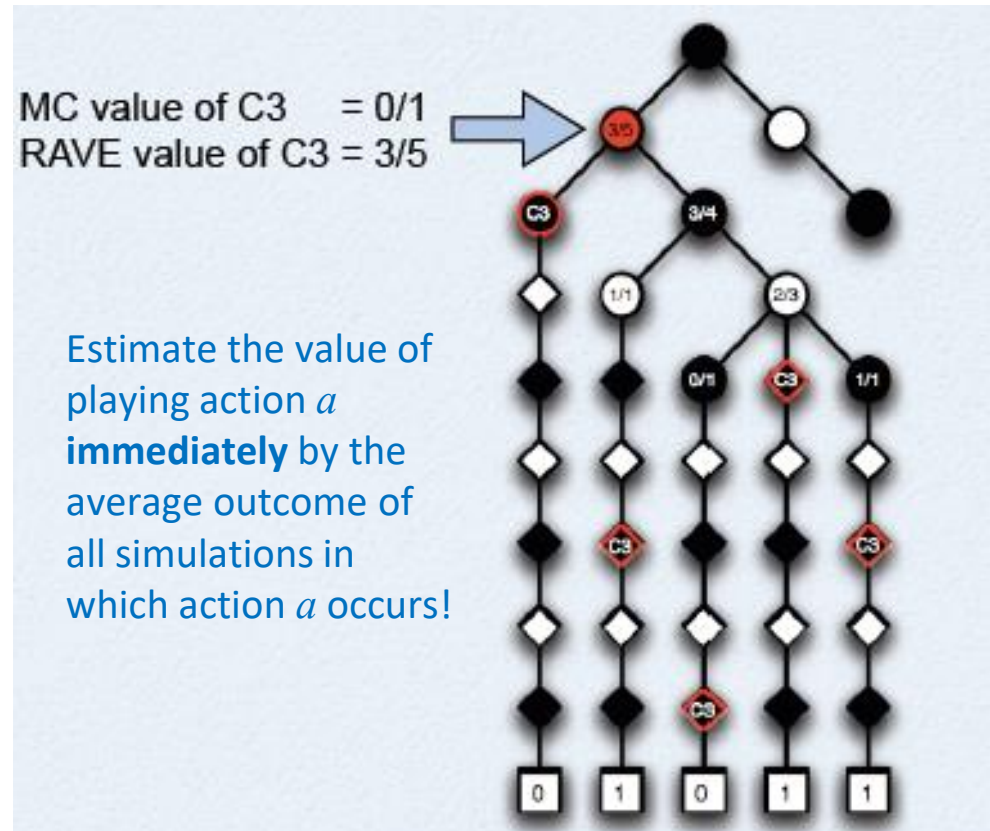
Rapid Action-Value Estimation (RAVE)

- Can we *share knowledge* among related nodes?
- **Assumption:** the value of a move is unaffected by moves played elsewhere on the board



Rapid Action-Value Estimation (RAVE)

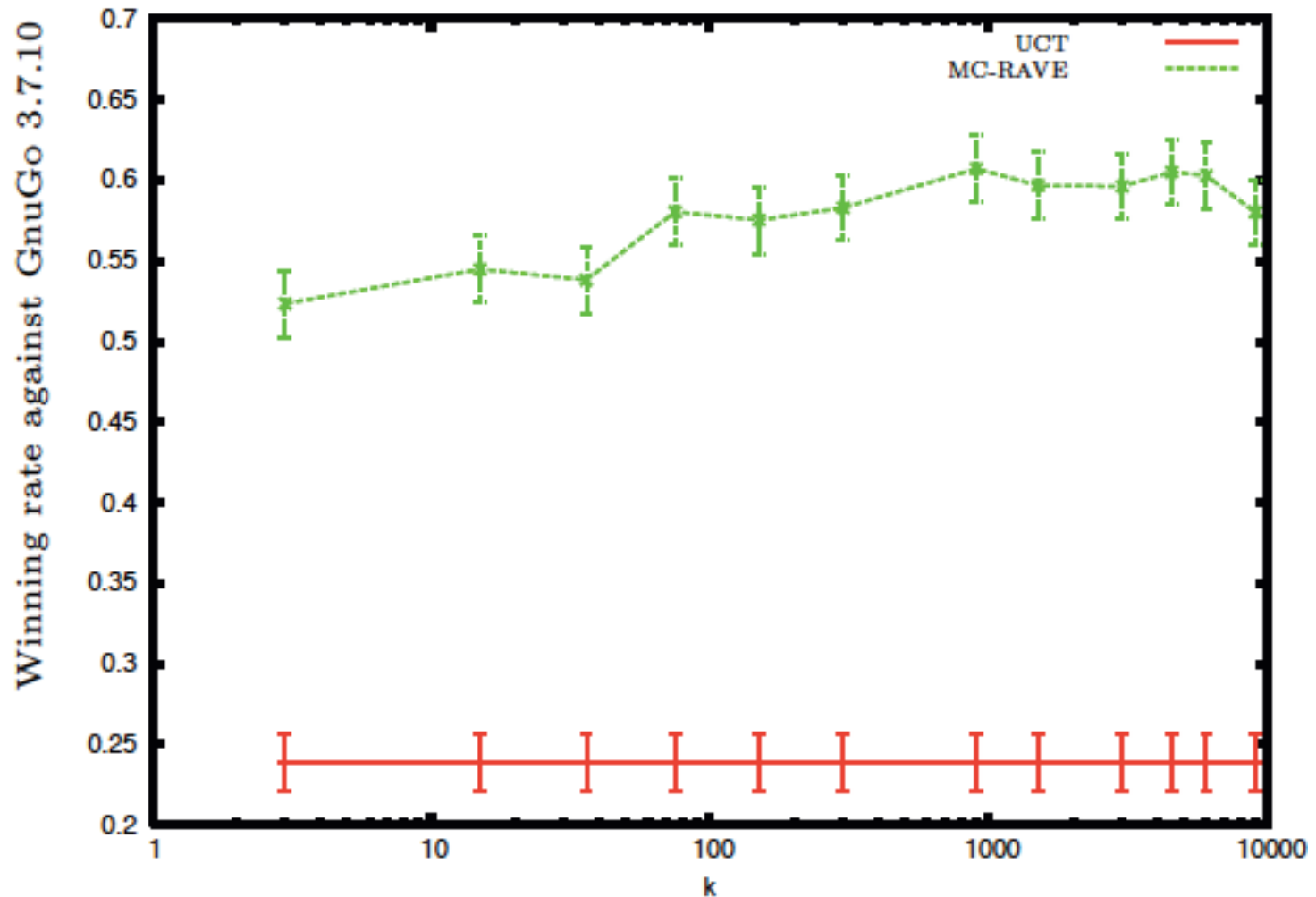
- Can we *share knowledge* among related nodes?
- **Assumption:** the value of a move is unaffected by moves played elsewhere on the board
- RAVE heuristic provides more information: a move can appear several times over a handful of simulations
- RAVE shares knowledge across nodes and gives a rapid, **biased** value estimate



State Representation Matters!

- **Original assumption:** each move at each position in the board has its quality estimated separately
 - This doesn't make use of commonalities between board positions
 - As a result, we need many simulations for each position
- **RAVE assumption:** only the move itself matters
 - Simplifies state space, requiring fewer simulations to get good estimates
 - Might oversimplify, leading to inaccurate estimates
- This trade-off between **model complexity** and **representational power** is pervasive in statistical estimation!

MC-RAVE in MoGo (2007)



MoGo (2007)

- MoGo = heuristic MCTS + MC-RAVE + handcrafted default policy
- 99% winning rate against best traditional programs
- Gold medal at Computer Go Olympiad
- First victory against professional players (on a 9x9 board)

AlphaGo (2016)

- Based on advances in state representation, value estimation, policy learning using **deep neural networks**



- Victory (4-1) over Lee Sedol on the full 19x19 board

Monte-Carlo tree search (vs α - β)

Advantages

- Not as pessimistic
- Converges to the Minimax solution in the limit
- It's an **anytime** algorithm: its performance improves with number of lines of play
- Less affected by the branching factor:
 - we get to the end of the game simulating one branch only
 - we can control the number of lines of play to match the available time
 - if branching factor is huge, search can go much deeper, which is a big gain
- Easy to parallelize

Disadvantages

- May miss optimal play (because it won't see all moves at deeper nodes)
- Policy used to generate candidate plays is very important.
- E.g., can use an opponent model, or just simple randomization

Recap Questions

- **L6** discussed minimax search for two-player, perfect information games
 - What is a two player zero-sum game?
 - What assumptions does it make about how your opponent will behave?
- **L7** talked about ways to prune the search tree
 - Why does alpha-beta pruning always give the minimax answer?
 - What is an evaluation function, and its role in these algorithms?
- **L8** used random simulations for learning to play games
 - Know the difference between tree and default policies
 - Think about what inputs are needed from the programmer
 - When do you expect MCTS to do well? Poorly?
 - If we have code for another strong solution, can we use it within MCTS?